



اَوْنُوْرَسِيْتِي تِكْنُوْلُوْجِي مَآرَا
UNIVERSITI
TEKNOLOGI
MARA

Cawangan Perak
Kampus Tapah

ICT200

INTRODUCTION TO DATABASE MANAGEMENT

GROUP PROJECT :

FINAL REPORT

Food Ordering Database System

Nasi Kandar Cik Yah

Lecturer : PUAN NOR AZZYATI BINTI HASHIM

Prepared by:

Minions

STUDENT NAME	STUDENT ID	CLASS GROUP
NURUL AIESYAH FATIHAH BINTI NORAZEHASRAM	2024225316	3F
SHANNAH LOVE OLAS	2024237256	3F
NORSYARIDA BINTI ROSLI	2024205848	3F
ANA KHAIRUN NISA' BINTI MOHD ZAIDI	2024296826	3F

TABLE OF CONTENTS

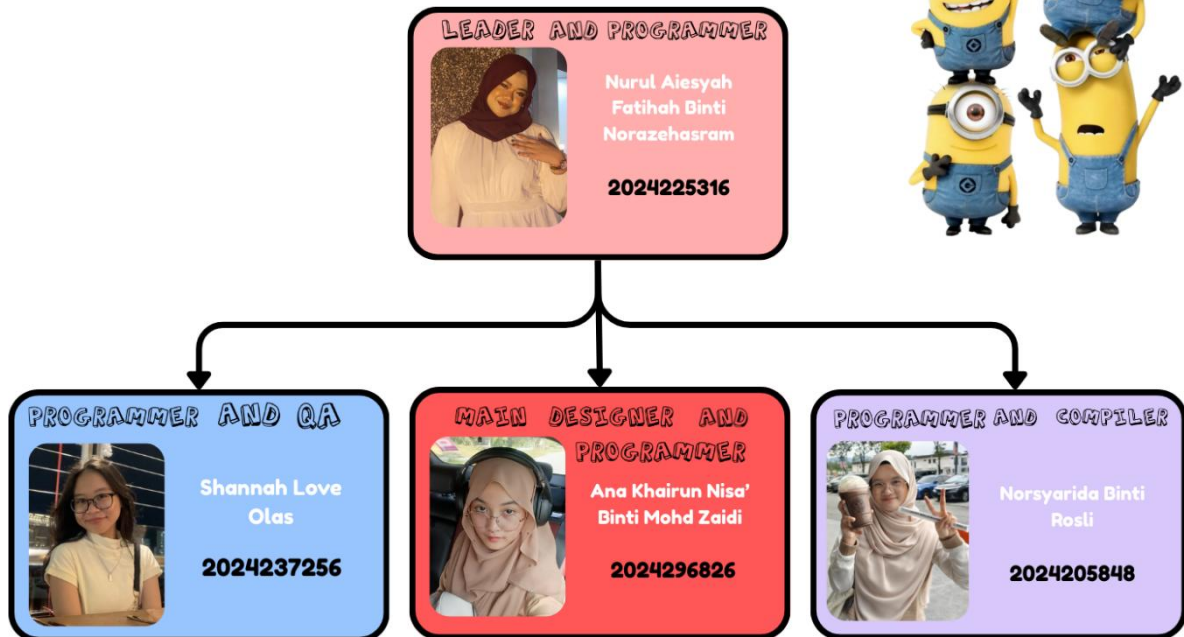
	Page
1.0 BUSINESS REQUIREMENTS	
1.1 Company Background	1 – 2
1.2 Company Organization Chart	3
1.3 Description of Daily Operation and Data Needs	4 – 5
1.4 Problems Findings in Data Management	6
1.5 Objectives of the proposed database System	7
1.6 Requirement for the database System	8 – 10
2.0 CONCEPTUAL DATABASE DESIGN	
2.1 Business Rules Findings	11
2.2 Entities and attributes Findings	12
2.3 ERD Design	13
3.0 LOGICAL DATABASE DESIGN	
3.1 Relational Schema	14
3.2 Data Dictionary	15 – 16
4.0 DATABASE IMPLEMENTATION	17 – 29
5.0 REFERENCES	30 – 31
6.0 APPENDIX	
6.1 Appendix A	32 – 41
6.2 Appendix B	42 – 49
7.0 RUBRIC	50 - 54

LIST OF FIGURES

		Page
Figure 1	Student's Profile	iii
Figure 2	Nasi Kandar Cik Yah Storefront View	1
Figure 3	Company Organization Chart	3
Figure 4	Restaurant Menu Part 1	5
Figure 5	Restaurant Menu Part 2	5
Figure 6	Restaurant Menu Part 3	5
Figure 7	ERD	13
Figure 8	Evidence 1 of Interview Session	42
Figure 9	Evidence 2 of Interview Session	43
Figure 10	Group Members with the Representative of the Company	44

STUDENT PROFILE

The Minions



**Nurul Aiesyah Fatimah Binti Norazehasram
(2024225316)**

Middle School Course: Science Stream

What's my ambition: Technology Consultant

Middle School Name: SMK Taman Desaminium, Seri Kembangan, Selangor

**Shannah Love Olan
(2024237256)**

Middle School Course: Accounting, Economics and Business.

What's my ambition: Doctor or Radiographer

Middle School Name: SM St. Dominic, Sabah.

**Ana Khairun Nisa' Binti Mohd Zaidi
(2024296826)**

Middle School Course: Literature Geography and Perniagaan

What's my ambition: Backend Developer

Middle School Name: Sekolah Agama Menengah Tinggi Sultan Salahuddin Abdul Aziz Shah, Selangor

**Norsyarida Binti Rosli
(2024205848)**

Middle School Course: Technical Science

What's my ambition: Architect

Middle School Name: SMK Alang Iskandar, Bagan Serai, Perak.

Figure 1 : Student's Profile

1.0 BUSINESS REQUIREMENT

1.1 Company Background



Figure 2 : Nasi Kandar Cik Yah Storefront View

Nasi Kandar Cik Yah is a local eatery operated by Mrs. Rokiyah binti Baharun, fondly known as Cik Yah. Although the restaurant officially opened in 2021 at Tapah, Perak, Cik Yah's entrepreneurial journey began much earlier. She first ventured into the food business in 2007 in Terengganu, where she took over a shop previously run by a relative. In 2017, she relocated to Tapah after receiving a tender to operate within a university area. Her experience continued to grow as she managed stalls in several locations, including the administration centre and Gamma, before eventually establishing her own independent restaurant outside the campus in 2023.

The inspiration behind serving Nasi Kandar came from her husband, who encouraged her to introduce the dish after noticing its popularity and potential. With no traditional family recipe, both husband and wife developed their own version through observation, tasting and experimentation, which eventually became the signature identity of the business.

Today, Nasi Kandar Cik Yah offers a variety of dishes including Nasi Kandar meals, affordable daily set options for students, and breakfast items such as roti canai and nasi lemak. Its generous portions and reasonable prices make it a favourite among local residents and university students. Prices fluctuate based on daily market conditions, particularly for ingredients such as fish and squid.

The restaurant is supported by a small team of full-time workers who handle responsibilities such as cooking, preparing drinks, managing ingredients and maintaining overall cleanliness. It operates daily from 7:30 a.m. to 4:00 p.m., with lunchtime being the peak period.

Guided by its business vision, Nasi Kandar Cik Yah strives to provide affordable, high-quality and freshly prepared meals while ensuring customer satisfaction through fast and friendly service. Its mission is to modernize restaurant operations by adopting a computerized database system that can enhance efficiency, minimize errors and strengthen record management. The main objective of this digital improvement is to ensure that all sales, order and stock information is accurately stored, systematically organized and easily accessible to support effective business decision-making.

Looking ahead, Cik Yah hopes to sustain the business in the long term and eventually pass it down to her children, preserving the family's legacy while continuing to serve the community with good food and warm hospitality.

Company Information:

Business Name: Nasi Kandar Cik Yah

Owner: Mrs. Rokiyah Binti Baharun

Location: Tapah, Perak, Malaysia

Shop Contact Number: 05-4035678

Operating Hours: 7:30 a.m. – 4:00 p.m. (daily)

1.2 Company Organization Chart

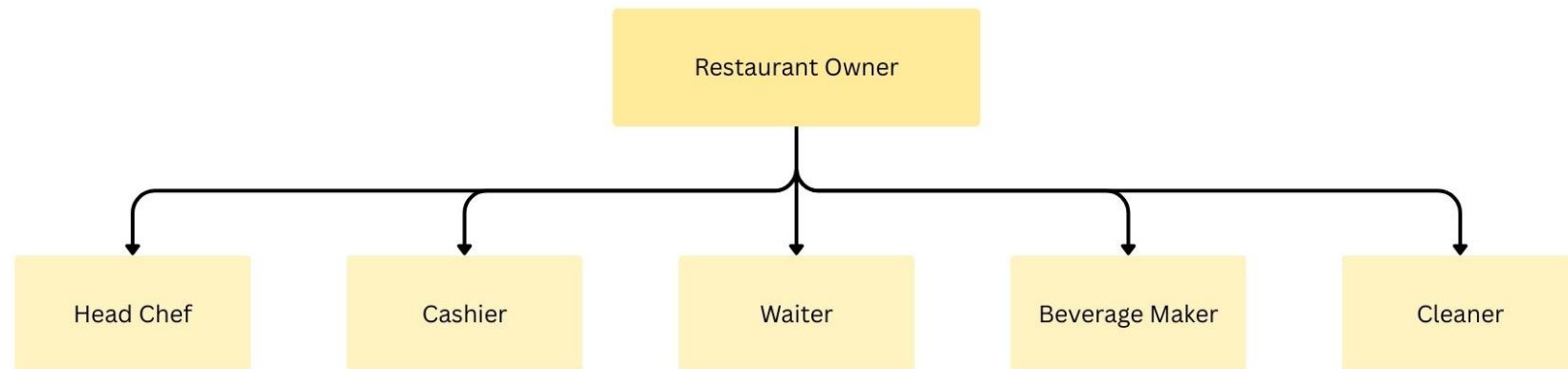


Figure 3 : Company Organization Chart

1.3 Description of Daily Operation and Data Needs

The daily operation of Nasi Kandar Cik Yah involves three main business activities: order taking, payment processing, and sales report generation. These activities are performed daily to serve customers and manage restaurant transactions. Through these routine operations, various types of operational data are created and needed to support accurate order handling, payment recording, and sales monitoring.

Order taking begins when staff first ask customers for the order type, either dine-in, takeaway, or pick-up. For dine-in orders, the table number is recorded, while order remarks are written for pick-up orders or special customer requests. Staff then record the selected food items and quantities, and the order status is updated manually to inform the kitchen staff about the progress of each order. The order data needed include order ID, order date, order type, table number for dine-in orders, order remarks for pick-up or special requests, and order status for the cook to ensure proper order preparation and tracking.

Payment processing takes place after an order is completed. Staff calculate the total amount payable using a calculator, and customers make payments either in cash or through QR bank transfer. The payment is recorded together with the related order details and the staff member handling the transaction. The payment data needed include sales ID, sales date, total amount, and payment method, which are required to record each transaction accurately and monitor daily income.

Sales report generation is performed by recording daily sales manually in record books. Individual sales transactions are summarized to determine total daily sales and overall business performance. This activity relies on accumulated sales information to produce summaries for review and reference. The sales data needed include sales date and total sales amount, which are required to support daily sales reporting and basic financial analysis.

Overall, these routine business activities generate interconnected data related to orders, payments, sales, and staff responsibilities, which are essential for managing daily restaurant operations efficiently and maintaining accurate business records.

ROTI CANAI		Minuman			
ROTI	HARGA	NESLO	4.50	PANAS	SEJUK
Roti Kosong	RM 1.50	Teh / Keladi	RM 2.00	RM 2.50	RM 2.50
Roti Telur	RM 2.50	Teh O	RM 1.50	RM 2.00	RM 2.00
Roti Planta	RM 2.50	Kopi O	RM 2.00	RM 2.50	RM 2.50
Roti Bom	RM 2.50	Kopi O	RM 1.50	RM 2.00	RM 2.00
Roti Sarang	RM 3.50	Nesclle	RM 3.00	RM 3.50	RM 3.50
Roti Jantan	RM 3.50	Nesclle O	RM 2.50	RM 3.00	RM 3.00
Roti Kahwin	RM 4.00	Milo	RM 3.00	RM 3.50	RM 3.50
Roti Sardin	RM 4.00	Milo O	RM 2.50	RM 3.00	RM 3.00
Roti Tiau	RM 3.00	Hortick	RM 3.00	RM 3.50	RM 3.50
Roti Pisang	RM 3.00	Sirap	-	RM 2.00	RM 2.00
Roti Milo	RM 3.00	Sanquik	RM 2.00	RM 2.50	RM 2.50
Ceklat	RM 3.00	Extra Joss	-	RM 3.00	RM 3.00
Ceklat Pisang	RM 4.00	Limau Asamoi	-	RM 2.50	RM 2.50
Roti Tampil	RM 2.50	Bandung	-	RM 2.50	RM 2.50
		Sirap Limau	-	RM 2.50	RM 2.50
		Teh O Ais Limau	-	RM 3.00	RM 3.00
		Teh O Ais Lemon	2.50	RM 3.00	RM 3.00
		REMARKS - BUNGKUS DLM PLASTIK + RM0.50			

Figure 4 : Restaurant Menu Part 1

NASI KANDOO* CIK YAH MENU MAKANAN		NASI KANDOO* CIK YAH MENU HARIAN		MENU MAKANAN	
SET A. RM6.00	SET B. RM7.50	7.00 PAGI	10.00 PAGI		HARGA
NASI	NASI	ROTI CANAI	MEEHOON GORENG	NASI PUTH	RM 1.50
ATAM	ATAM	NASI LEMAK	NASI KANDAR	AYAM MADU	RM 4.00
TELUR MASIN	TELUR			AYAM BAWANG	RM 4.00
				AYAM GORENG	RM 4.00
				IKAN MERAH	RM 6.00 / 10/25
				UDANG	RM 3.00
				SITONG	RM 6.00
				TELUR SUTONG	RM 3.00
				TELUR KARI	RM -
				TELUR REBUS	RM 1.00
				TELUR MASIN	RM 1.00
				PAPADOM	RM 1.50
				PERIA GORENG	RM 0.50
				SAYUR KOBIS	RM 1.00
				SAYUR TALOH	RM 1.00
				BENDI	RM 0.50 / 1.50
				OLU HENAU	RM 0.50
				SAMBAL KELAPA	RM 0.50
				Sambal belacan	RM 0.50

Figure 5 : Restaurant Menu Part 2

KARI KAMBING	
RM	7.00 1 PIRING
TEL KEDAI - 05-4035678	

Figure 6 : Restaurant Menu Part 3

1.4 Problems Findings in Data Management

From the interview conducted, several data-management problems were identified in the current manual system:

- 1. Inefficient Record Management:** The manual system often results in incomplete transaction records, order mix-ups during peak lunch hours, and time-consuming data retrieval, as sales and purchase information must be searched through handwritten record books, leading to inaccurate and inefficient data management.
- 2. Calculation Errors:** Daily income and payments are calculated using a calculator, which often leads to repeated checks and inconsistent figures.
- 3. No Automated Sales Summary:** The owner no longer prepares monthly reports because manual calculation takes too much time.
- 4. Lack of Real-Time Order Tracking:** Verbal and paper-based order recording makes it difficult to monitor order status, increasing the likelihood of missed or duplicated orders.

1.5 Objectives of the proposed database System

The objectives of this database are:

- i. To identify requirements for food ordering and sales management at Nasi Kandar Cik Yah.
- ii. To design a database that can manage food orders, menu items, payments, and sales records.
- iii. To develop a database for the Nasi Kandar Ordering Database Management System.

1.6 Requirement for the database system

1. User Authentication Login & Staff Management

Related ERD Entity: *Staff (Staff_ID)*

Purpose / Features:

- Secure login for the system owner and staff members.
- Role-based access control (e.g., admin and normal staff).
- Insert, update, and delete staff records, including adding new staff, updating staff details, and removing inactive staff.
- Reset staff passwords when required.

Importance:

This module ensures that only authorized users can access or modify the system. Proper access control protects data integrity and prevents unauthorized changes to orders, sales records, menu items, and reports.

2. Menu Management

Related ERD Entity: *Menu (Menu_ID)*

Purpose / Features:

- Insert new menu items into the system.
- Update existing menu information such as menu name, price, category, and description.
- Delete menu items that are no longer sold.
- Mark menu items as unavailable when sold out.

Importance:

This module ensures that menu information remains accurate during order processing. Updated menu data helps prevent pricing errors and ensures customers are charged correctly based on current availability.

3. Order Management & Order Tracking

Related ERD Entities: *Orders (OrderID)*, *OrderMenu*, *Menu*

Purpose / Features:

- Insert new customer orders into the system.
- Insert ordered items using the OrderMenu entity, including:
 - OrderID (PK, FK)
 - MenuID (PK, FK)
 - Quantity
 - Subtotal
 - Request
- Update order details and order status during preparation and service.
- Delete or cancel orders if required.
- Support different order types such as dine-in, takeaway, and pick-up.
- Track order status for kitchen preparation and service.

Importance:

This module ensures that all customer orders are recorded accurately. It reduces order confusion, supports proper food preparation, and maintains structured order records.

4. Sale and Payment Processing

Related ERD Entity: *Sales (Sales_ID)*

Purpose / Features:

- Insert sales records linked to completed orders.
- Update payment information such as payment method or total amount if corrections are needed.
- Support multiple payment methods such as cash and QR bank transfer.
- Store total sales amount and payment details for each transaction.

Importance:

This module ensures accurate recording of payments and sales transactions. It reduces manual calculation errors and supports reliable financial tracking.

5. Sales Reporting Summary

Related ERD Entities: *Sales, Staff*

Purpose / Features:

- Generate daily, weekly, and monthly sales summaries.
- Provide sales analysis based on date and staff handling transactions.
- Retrieve stored sales data for reporting and review.

Importance:

This module eliminates the need for manual sales calculations, supporting better business decision-making.

6. Search & Retrieval

Related ERD Entities: *Orders, Sales, Menu, Staff*

Purpose / Features:

- Search for past order records based on order type, table number, or order status.
- Search for past sales records.
- Search for menu items and display their details when needed.
- Search for staff records involved in handling orders and sales.
- Retrieve complete order, sales, menu, and staff information efficiently.

Importance:

This module speeds up data retrieval and reduces reliance on handwritten records, improving efficiency and accuracy.

2.0 CONCEPTUAL DATABASE DESIGN

2.1 Business Rules Findings

A **staff** may work for many **hours**

An **hour** is worked by only one **staff**

A **staff** member can generate many **sales**.

A **sale** must be generated by one **staff** member.

The **Generates** table serves for bridge entity.

A **sale** has many **sales line**

A **sales line** belongs to one **sale**

Every **receipt** must have exactly one **sale**.

Every **sale** must belong to one **receipt**.

An **order** can contain many **sales line**

A **sales line** belongs to one **order**

A **menu** has many **sales line**

A **sales line** belongs to one **menu**

A **staff** can record many **expenses**.

Each **expense** is recorded by exactly one **staff**.

2.2 Entities and Attributes Findings

ENTITY	ATTRIBUTES
STAFF	- <u>StaffID</u> (PK) - StaffName - StaffPhoneNo - StaffRole - StaffUsername - StaffPassword
ORDERS	- <u>OrderID</u> (PK) - OrderDate - OrderType - TableNo - OrderRemark - OrderStatus
MENU	- <u>MenuID</u> (PK) - MenuName - MenuPrice - MenuCategory - MenuDesc
ORDERMENU	- <u>MenuID*</u> (PK, FK1) - <u>OrderID*</u> (PK, FK2) - Quantity - Subtotal - Request
SALES	- <u>SalesID</u> (PK) - SalesDate - SalesTotal - SalesPayMethod - OrderID* (FK) - StaffID* (FK)

2.3 Entity Relationship Diagram

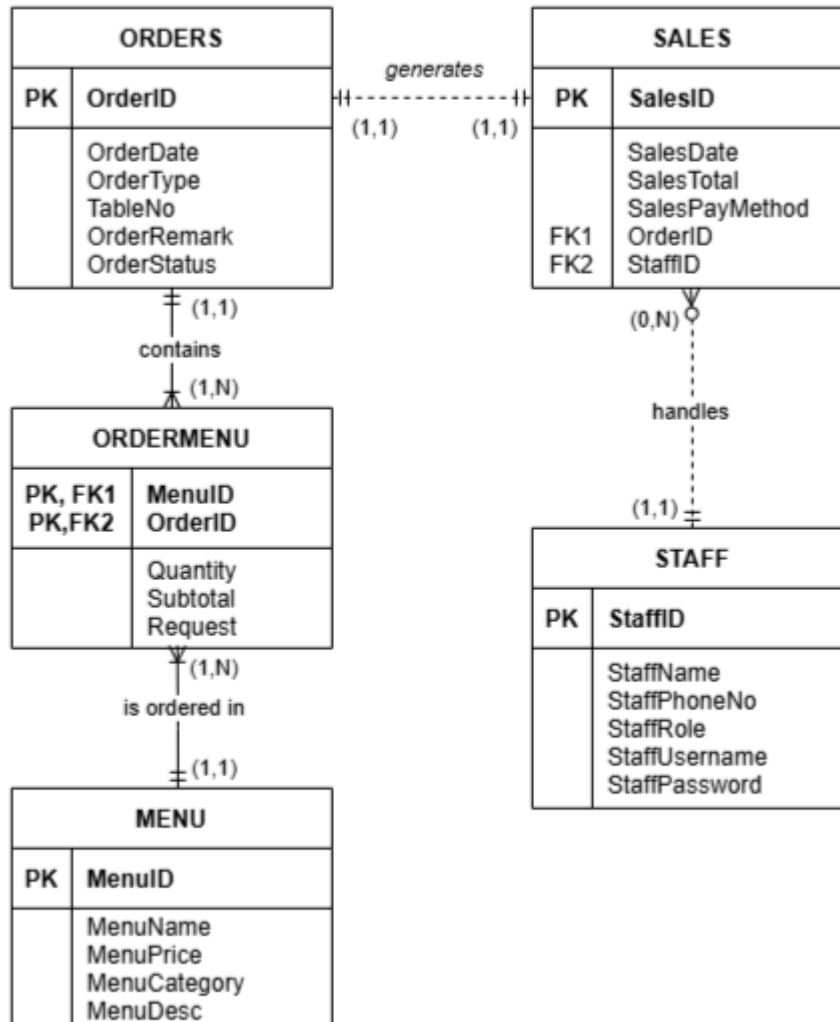


Figure 7 : ERD

3.0 LOGICAL DATABASE DESIGN

3.1 Relational Schema

SALES (**SalesID**, SalesDate, SalesTotal, SalesPayMethod, OrderID*, StaffID*)

STAFF (**StaffID**, StaffName, StaffPhoneNo, StaffRole, StaffUsername, StaffPassword)

ORDERS (**OrderID**, OrderDate, OrderType, TableNo, OrderRemark, OrderStatus)

MENU (**MenuID**, MenuName, MenuPrice, MenuCategory, MenuDesc)

ORDERMENU (**MenuID***, **OrderID***, Quantity, Subtotal, Request)

3.2 Data Dictionary

Table Name	Attribute Name	Contents	Data Type	Format	Range	Required	PK or FK	PK Referenced Table
STAFF	StaffID	Staff ID	VARCHAR(4)	XXXX	NA	YES	PK	
	StaffName	Staff Name	VARCHAR(50)	XXXXXXXXXXXXXXXXXXXX	NA	YES		
	StaffPhoneNo	Staff Phone Number	VARCHAR(12)	XXXXXXXXXXXXXXXXXXXX	NA	YES		
	StaffRole	Staff Role	VARCHAR(20)	XXXXXXXXXXXXXXXXXXXX	NA	NO		
	StaffUsername	Staff Username	VARCHAR(20)	XXXXXXXXXXXXXXXXXXXX	NA	YES		
	StaffPassword	Staff Password	VARCHAR(20)	XXXXXXXXXXXXXXXXXXXX	NA	YES		
ORDERS	OrderID	Order ID	VARCHAR(10)	XXXXXXXXXXXXXXXXXXXX	NA	YES	PK	
	OrderDate	Order Date Taken	DATE	DD-MM-YY	NA	YES		
	OrderType	Order Type	VARCHAR(20)	XXXXXXXXXXXXXXXXXXXX	Dine In, Take Away, Pick Up	YES		
	TableNo	Table Number	INT	####	NA	NO		
	OrderRemark	Order Remarks	VARCHAR(150)	XXXXXXXXXXXXXXXXXXXX	NA	NO		
	OrderStatus	Order Status	VARCHAR(10)	XXXXXXXXXXXX	Preparing, Ready, Completed	NO		
SALES	SalesID	Sales ID	VARCHAR(10)	XXXXXXXXXXXXXXXXXXXX	NA	YES	PK	
	OrderID	Order ID	VARCHAR(10)	XXXXXXXXXXXXXXXXXXXX	NA	YES	FK	ORDERS
	StaffID	Staff ID	VARCHAR(4)	XXXX	NA	YES	FK	STAFF
	SalesDate	Sales Date	DATE	DD-MM-YY	NA	YES		
	SalesTotal	Total Sales	NUMERIC(6,2)	####.##	0.00 - 0000.00	YES		
	SalesPayMethod	Sales Payment Method	VARCHAR(20)	XXXXXXXXXXXXXXXXXXXX	Cash, E-Wallet	YES		
MENU	MenuID	Menu ID	VARCHAR(10)	XXXXXXXXXX	NA	YES	PK	
	MenuName	Menu Name	VARCHAR(50)	XXXXXXXXXXXXXXXXXXXX	NA	YES		

ORDERMENU	MenuCategory	Menu Category	VARCHAR(20)	XXXXXXXXXXXXXXXXXXXX	NA	NO		
	MenuPrice	Menu Price	NUMERIC(6,2)	####.##	0.00 - 0000.00	YES		
	MenuDesc	Menu Description	VARCHAR(50)	XXXXXXXXXXXXXXXXXXXX	NA	NO		
	MenuID	Menu ID	VARCHAR(10)	XXXXXXXXXXXXXXXXXXXX	NA	YES	PK/FK	MENU
	OrderID	Order ID	VARCHAR(10)	XXXXXXXXXXXXXXXXXXXX	NA	YES	PK/FK	ORDERS
	Quantity	Quantity Ordered	INT	##	NA	YES		
	Subtotal	Subtotal	NUMERIC(6,2)	####.##	0.00 - 0000.00	YES		
	Request	Request	VARCHAR(50)	XXXXXXXXXXXXXXXXXXXX	NA	NO		

4.0 DATABASE IMPLEMENTATION

DDL - CREATE

- a. For each table, copy and paste the CREATE TABLE statement and then screenshot the DESCRIBE statement and its output result.

```
CREATE DATABASE CIKYAHDB;  
USE CIKYAHDB;
```

```
CREATE TABLE STAFF(  
STAFFID VARCHAR(4) PRIMARY KEY,  
STAFFNAME VARCHAR(50) NOT NULL,  
STAFFPHONENO VARCHAR(12) NOT NULL,  
STAFFROLE VARCHAR(20),  
STAFFUSERNAME VARCHAR(20) NOT NULL,  
STAFFPASSWORD VARCHAR(20) NOT NULL);
```

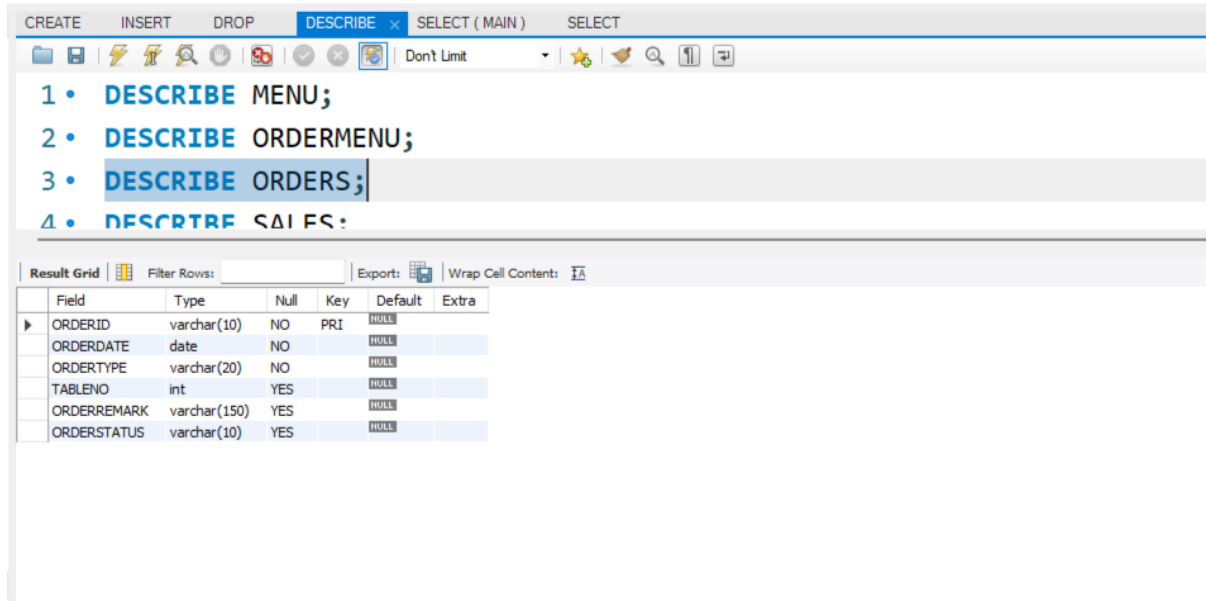
The screenshot shows a database management tool interface. The top menu bar includes 'CREATE', 'INSERT', 'DROP', 'DESCRIBE', 'SELECT (MAIN)', and 'SELECT'. The 'DESCRIBE' menu is open, showing a list of tables: '3 • DESCRIBE ORDERS;', '4 • DESCRIBE SALES;', '5 • DESCRIBE STAFF;', and '6'. The 'DESCRIBE STAFF;' option is selected. Below the menu, the 'Result Grid' is displayed, showing the output of the DESCRIBE statement for the STAFF table. The grid has columns: Field, Type, Null, Key, Default, and Extra. The data is as follows:

Field	Type	Null	Key	Default	Extra
STAFFID	varchar(4)	NO	PRI	NULL	
STAFFNAME	varchar(50)	NO		NULL	
STAFFPHONENO	varchar(12)	NO		NULL	
STAFFROLE	varchar(20)	YES		NULL	
STAFFUSERNAME	varchar(20)	NO		NULL	
STAFFPASSWORD	varchar(20)	NO		NULL	

```

CREATE TABLE ORDERS (
ORDERID VARCHAR(10) PRIMARY KEY,
ORDERDATE DATE NOT NULL,
ORDERTYPE VARCHAR(20) NOT NULL,
TABLENO INT,
ORDERREMARK VARCHAR(150),
ORDERSTATUS VARCHAR(10));

```



The screenshot shows a database management interface with a 'DESCRIBE' menu open. The menu items are:

- 1 • DESCRIBE MENU;
- 2 • DESCRIBE ORDERMENU;
- 3 • DESCRIBE ORDERS;
- 4 • DESCRIBE SALES;

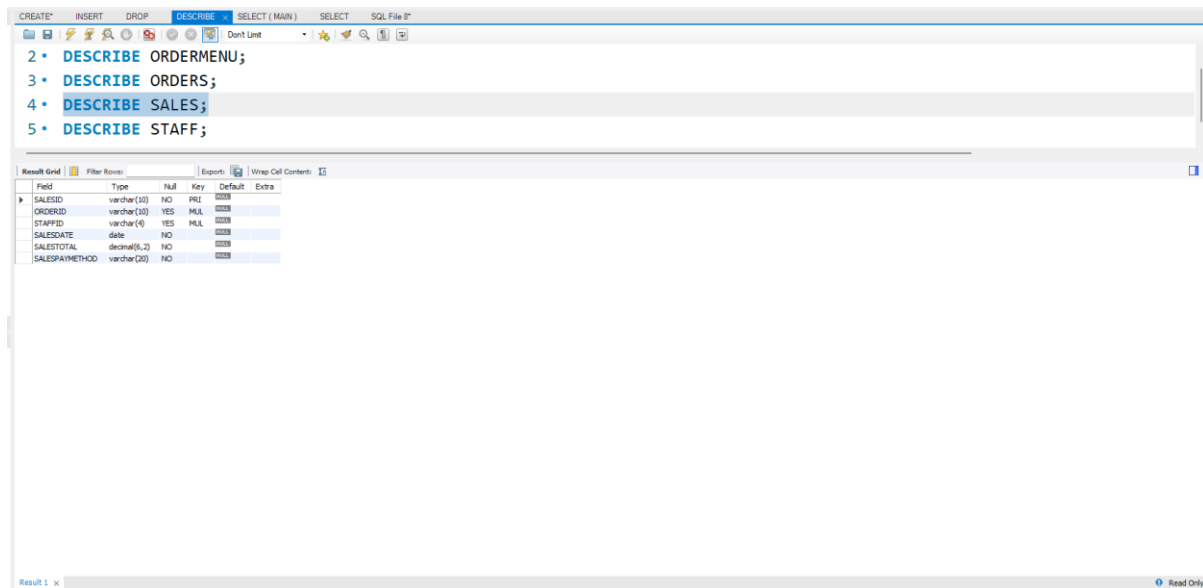
Below the menu, the 'Result Grid' displays the structure of the 'ORDERS' table:

Field	Type	Null	Key	Default	Extra
ORDERID	varchar(10)	NO	PRI	NULL	
ORDERDATE	date	NO		NULL	
ORDERTYPE	varchar(20)	NO		NULL	
TABLENO	int	YES		NULL	
ORDERREMARK	varchar(150)	YES		NULL	
ORDERSTATUS	varchar(10)	YES		NULL	

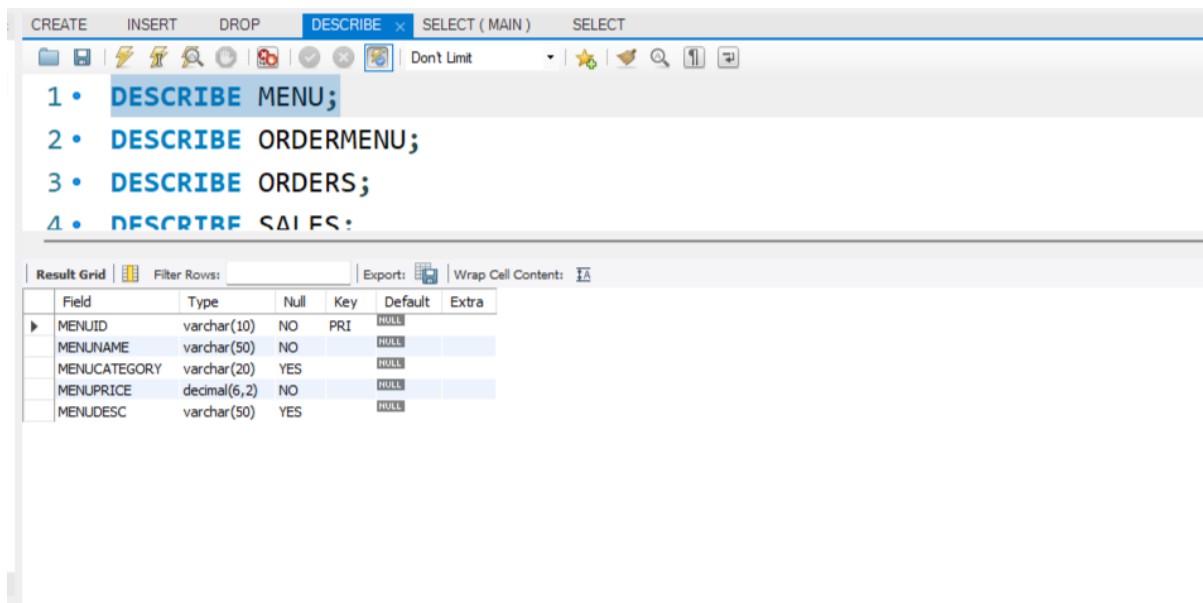
```

CREATE TABLE SALES (
SALESID VARCHAR(10) PRIMARY KEY,
ORDERID VARCHAR(10),
STAFFID VARCHAR(4),
SALESDATE DATE NOT NULL,
SALESTOTAL NUMERIC(6,2) NOT NULL,
SALESPAYMETHOD VARCHAR(20) NOT NULL,
INDEX (ORDERID),
INDEX (STAFFID),
CONSTRAINT FK_ORDERID FOREIGN KEY (ORDERID) REFERENCES
ORDERS (ORDERID),
CONSTRAINT FK_STAFFID FOREIGN KEY (STAFFID) REFERENCES
STAFF (STAFFID));

```



```
CREATE TABLE MENU (
MENUID VARCHAR(10) PRIMARY KEY,
MENUNAME VARCHAR(50) NOT NULL,
MENUCATEGORY VARCHAR(20),
MENUPRICE NUMERIC(6,2) NOT NULL,
MENUDESC VARCHAR(50));
```



```

CREATE TABLE ORDERMENU (
MENUID VARCHAR(10) ,
ORDERID VARCHAR(10) ,
QUANTITY INT NOT NULL,
SUBTOTAL NUMERIC(6,2) NOT NULL,
REQUEST VARCHAR(50) ,
INDEX (MENUID) ,
INDEX (ORDERID) ,
CONSTRAINT PK_ORDERMENU PRIMARY KEY (MENUID,ORDERID) ,
CONSTRAINT FK_MENUID FOREIGN KEY (MENUID) REFERENCES MENU (MENUID) ,
CONSTRAINT FK_ORDERID_ORDERMENU FOREIGN KEY (ORDERID) REFERENCES
ORDERS (ORDERID) ) ;

```

The screenshot shows a database management interface with a menu bar (CREATE, INSERT, DROP, DESCRIBE, SELECT (MAIN), SELECT) and a toolbar. Below the menu, a list of SQL commands is displayed:

- 1 • DESCRIBE MENU;
- 2 • DESCRIBE ORDERMENU;
- 3 • DESCRIBE ORDERS;
- 4 • DESCRIBE SALES;

Below the commands, there is a "Result Grid" section with a table showing the structure of the ORDERMENU table:

Field	Type	Null	Key	Default	Extra
MENUID	varchar(10)	NO	PRI	NULL	
ORDERID	varchar(10)	NO	PRI	NULL	
QUANTITY	int	NO		NULL	
SUBTOTAL	decimal(6,2)	NO		NULL	
REQUEST	varchar(50)	YES		NULL	

DML – SELECT QUERY

- b. 1 SQL statement that list significant information by selecting certain columns from a single table in the database. (Basic Select)

```
CREATE INSERT DROP DESCRIBE SELECT / MAIN Y SELECT* SQL File B*
2 /* QUERY : To display a list of menu name, category, price and description for customer view */
3
4 • SELECT MENUNAME, MENCATEGORY, MENUPRICE, MENUDESC
5 FROM MENU;
6
```

MENUNAME	MENCATEGORY	MENUPRICE	MENUDESC
TELUR REBUS	ADD-ON	1.50	
BEUCE	ADD-ON	0.50	
PAPADOM	ADD-ON	1.50	
TEMPE GORENG	ADD-ON	0.50	
AYAM GORENG	LAUK	4.00	SATU PIRING
AYAM KAKI	LAUK	4.00	SATU PIRING
AYAM KICAP	LAUK	4.00	SATU PIRING
IKAN MERAH	LAUK	6.00	SATU KECIL
IKAN GORENG	LAUK	3.00	SEKOR
SOTONG	LAUK	6.00	SATU BESAR
KAKI KAMENG	LAUK	7.00	SATU PIRING
TEH	MENUMAN	2.00	PANAS
TEH	MENUMAN	2.50	SEJUK
KOPET	MENUMAN	2.50	SEJUK
NGLO	MENUMAN	3.50	SEJUK
BANDUNG	MENUMAN	2.50	SEJUK
ROTI KOSONG	ROTI CANAI	1.50	
ROTI TELUR	ROTI CANAI	2.50	
ROTI BOM	ROTI CANAI	2.50	
ROTI KAHWIN	ROTI CANAI	4.00	
ROTI SARDEN	ROTI CANAI	4.00	
ROTI MELO	ROTI CANAI	3.00	
SET A	SET	8.50	NASI+AYAM+SAYUR+TELUR MASIN
SET B	SET	7.50	NASI+AYAM+SAYUR
SET C	SET	6.50	NASI+AYAM

- c. 1 SQL statement that list significant information from a single table in the database with SQL function (Numeric/String/Date&Time). (Basic Select)

```
CREATE INSERT DROP DESCRIBE SELECT / MAIN Y SELECT
9 /* QUERY: To display all sales records that occurred in January 2026
10 by filtering the sales date using the MONTH() and YEAR() function. */
11
12 • SELECT *
13 FROM SALES
14 WHERE MONTH(SALESDATE)= 01
15 AND YEAR(SALESDATE) = 2026;
```

SALESID	ORDERID	STAFFID	SALESDATE	SALESTOTAL	SALESPAYMENTMETHOD
SAL001	ORD001	CS01	2026-01-03	5.00	CASH
SAL002	ORD002	CS01	2026-01-03	10.00	E-WALLET
SAL003	ORD003	CS02	2026-01-03	8.50	CASH
SAL004	ORD004	CS02	2026-01-03	5.50	E-WALLET
SAL005	ORD005	CS01	2026-01-03	5.50	CASH
SAL006	ORD006	CS02	2026-01-04	6.00	CASH
SAL007	ORD007	CS01	2026-01-04	7.50	E-WALLET
SAL008	ORD008	CS02	2026-01-04	5.00	E-WALLET
SAL009	ORD009	CS01	2026-01-04	8.50	CASH
SAL010	ORD010	CS02	2026-01-04	3.00	CASH
SAL011	ORD011	CS01	2026-01-05	10.00	E-WALLET
SAL012	ORD012	CS02	2026-01-05	4.50	CASH
SAL013	ORD013	CS01	2026-01-05	4.50	E-WALLET
SAL014	ORD014	CS02	2026-01-05	8.50	E-WALLET
SAL015	ORD015	CS01	2026-01-05	11.00	CASH
SAL016	ORD016	CS02	2026-01-05	2.50	CASH
SAL017	ORD017	CS01	2026-01-05	8.00	E-WALLET
SAL018	ORD018	CS02	2026-01-05	9.50	E-WALLET
SAL019	ORD019	CS01	2026-01-06	19.50	E-WALLET
SAL021	ORD021	CS02	2026-01-06	19.00	CASH
SAL023	ORD023	CS02	2026-01-06	6.50	E-WALLET

- d. 1 SQL statement from a single table that uses comparison operator in the database. (Basic Select)

The screenshot shows a SQL IDE with a query editor and a result grid. The query is designed to display menu items with a price less than or equal to RM6.50. The result grid shows 26 rows of data.

```
20 /* QUERY : To display menu items with a price less than or equal to
21          RM6.50 in order to identify affordable menu options for budget-conscious customers. */
22
23 * SELECT MENUName, MENUPrice, MENUCategory, MENUDESC
24 FROM MENU
25 WHERE MENUPrice <= 6.50;
26
```

MENUName	MENUPrice	MENUCategory	MENUDESC
TELUR REBUS	1.50	ADD-ON	TELUR
BENICE	0.50	ADD-ON	TELUR
PAPADOM	1.50	ADD-ON	TELUR
TEMPE GORENG	0.50	ADD-ON	TELUR
AYAM GORENG	4.00	LAUK	SATU PIRING
AYAM KARI	4.00	LAUK	SATU PIRING
AYAM KECAP	4.00	LAUK	SATU PIRING
IKAN MERAH	6.00	LAUK	SAIZ KECIL
IKAN GORENG	3.00	LAUK	SEKOR
SOTONG	6.00	LAUK	SAIZ BESAR
TEH	2.00	MENUMAN	PANAS
TEH	2.50	MENUMAN	SEJUK
Kopi	2.50	MENUMAN	SEJUK
MILO	3.50	MENUMAN	SEJUK
SARDIN	2.50	MENUMAN	SEJUK
ROTI KOSONG	1.50	ROTI CANAI	TELUR
ROTI TELUR	2.50	ROTI CANAI	TELUR
ROTI BOM	2.50	ROTI CANAI	TELUR
ROTI KAHWIN	4.00	ROTI CANAI	TELUR
ROTI SARDIN	4.00	ROTI CANAI	TELUR
ROTI MILO	3.00	ROTI CANAI	TELUR
SET C	6.50	SET	NASIHATAM

- e. 1 SQL statement from a single table that uses LIKE clause in the database in ascending order. (Basic Select)

The screenshot shows a SQL IDE with a query editor and a result grid. The query is designed to display menu items that start with the word "ROTI" and sort them by menu price in ascending order. The result grid shows 6 rows of data.

```
29 /* QUERY : To display menu items that start with the word "ROTI" and sort them by menu price in
30          ascending order for easy price comparison. */
31
32 * SELECT MENUName, MENUPrice, MENUCategory
33 FROM MENU
34 WHERE MENUName LIKE 'ROTI%'
35 ORDER BY MENUPrice ASC;
36
```

MENUName	MENUPrice	MENUCategory
ROTI KOSONG	1.50	ROTI CANAI
ROTI TELUR	2.50	ROTI CANAI
ROTI BOM	2.50	ROTI CANAI
ROTI MILO	3.00	ROTI CANAI
ROTI KAHWIN	4.00	ROTI CANAI
ROTI SARDIN	4.00	ROTI CANAI

- f. 1 SQL statement from a single table that uses IN clause in the database in descending order.
(Basic Select)

```

39 /* QUERY: To display all pick-up and takeaway orders made on 6 January 2026,
40     including order details, sorted by order ID in descending order. */
41
42 * SELECT ORDERDATE, ORDERID, ORDERTYPE, ORDERSTATUS, ORDERREMARK
43 FROM ORDERS
44 WHERE ORDERTYPE IN ('PICK UP', 'TAKE-AWAY')
45 AND ORDERDATE = '2026-01-06'
46 ORDER BY ORDERID DESC;

```

ORDERDATE	ORDERID	ORDERTYPE	ORDERSTATUS	ORDERREMARK
2026-01-06	ORD023	TAKE-AWAY	COMPLETED	
2026-01-06	ORD021	TAKE-AWAY	COMPLETED	
2026-01-06	ORD020	PICK UP	READY	PICK UP AT 11AM BY AMR([145365362])

- g. 1 SQL statement to list significant information from many tables that has relationship in ascending order. (Join)

```

50 /* QUERY : To display receipts of complete order which includes menu, sales and staff details and
51     total items per order calculated using a subquery, sorted by order ID in ascending order. */
52
53 * SELECT O.ORDERID, ST.STAFFNAME, S.SALESID, M.MENUNAME, OM.QUANTITY, OM.SUBTOTAL, (SELECT SUM(OM2.QUANTITY) FROM ORDERMENU OM2 WHERE OM2.ORDERID = O.ORDERID) AS "TOTAL ITEMS" ,
54     S.SALESTOTAL AS "TOTAL PRICE", SALESDATE
55 FROM ORDERS O, ORDERMENU OM, MENU M, SALES S, STAFF ST
56 WHERE O.ORDERID = OM.ORDERID
57 AND OM.MENUID = M.MENUID
58 AND O.ORDERID = S.ORDERID
59 AND S.STAFFID = ST.STAFFID
60 ORDER BY O.ORDERID ASC;

```

ORDERID	STAFFNAME	SALESID	MENUNAME	QUANTITY	SUBTOTAL	TOTAL ITEMS	TOTAL PRICE	SALESDATE
ORD001	AHMAD SYAFIQ BIN YUSOF	SAL001	TEH	1	2.00	3	5.00	2026-01-03
ORD001	AHMAD SYAFIQ BIN YUSOF	SAL001	ROTI KOSONG	2	3.00	3	5.00	2026-01-03
ORD002	AHMAD SYAFIQ BIN YUSOF	SAL002	AYAM GORENG	1	4.00	3	10.00	2026-01-03
ORD002	AHMAD SYAFIQ BIN YUSOF	SAL002	MILO	1	3.50	3	10.00	2026-01-03
ORD002	AHMAD SYAFIQ BIN YUSOF	SAL002	ROTI TELUR	1	2.50	3	10.00	2026-01-03
ORD003	DAMIA IRDINA BENITI FIRDAUS	SAL003	SET A	1	8.50	1	8.50	2026-01-03
ORD004	DAMIA IRDINA BENITI FIRDAUS	SAL004	BECKE	3	1.50	4	5.50	2026-01-03
ORD004	DAMIA IRDINA BENITI FIRDAUS	SAL004	ROTI KAHWIN	1	4.00	4	5.50	2026-01-03
ORD005	AHMAD SYAFIQ BIN YUSOF	SAL005	BANDUNG	1	2.50	3	5.50	2026-01-03
ORD005	AHMAD SYAFIQ BIN YUSOF	SAL005	ROTI KOSONG	2	3.00	3	5.50	2026-01-03
ORD006	DAMIA IRDINA BENITI FIRDAUS	SAL006	AYAM KARI	1	4.00	2	6.00	2026-01-04
ORD006	DAMIA IRDINA BENITI FIRDAUS	SAL006	TEH	1	2.00	2	6.00	2026-01-04
ORD007	AHMAD SYAFIQ BIN YUSOF	SAL007	SET B	1	7.50	1	7.50	2026-01-04
ORD008	DAMIA IRDINA BENITI FIRDAUS	SAL008	ROTI BOM	2	5.00	2	5.00	2026-01-04
ORD009	AHMAD SYAFIQ BIN YUSOF	SAL009	IRAN MERAH	1	6.00	2	8.50	2026-01-04
ORD009	AHMAD SYAFIQ BIN YUSOF	SAL009	TEH	1	2.50	2	8.50	2026-01-04
ORD010	DAMIA IRDINA BENITI FIRDAUS	SAL010	ROTI MILO	1	3.00	1	3.00	2026-01-04
ORD011	AHMAD SYAFIQ BIN YUSOF	SAL011	MILO	1	3.50	2	10.00	2026-01-05
ORD011	AHMAD SYAFIQ BIN YUSOF	SAL011	SET C	1	6.50	2	10.00	2026-01-05
ORD012	DAMIA IRDINA BENITI FIRDAUS	SAL012	TEMPE GORENG	1	0.50	2	4.50	2026-01-05
ORD012	DAMIA IRDINA BENITI FIRDAUS	SAL012	ROTI SARDIN	1	4.00	2	4.50	2026-01-05
ORD013	AHMAD SYAFIQ BIN YUSOF	SAL013	ROTI KOSONG	3	4.50	3	4.50	2026-01-05
ORD014	DAMIA IRDINA BENITI FIRDAUS	SAL014	SOTONG	1	6.00	2	8.50	2026-01-05

- h. 1 SQL statement to list significant information from many tables that has relationship in descending order. (Join)

The screenshot shows a SQL IDE with a query window and a result grid. The query is as follows:

```
65 /* QUERY : To display sales ID, staff name and total sales amount in order to identify
66 high-value sales transactions handled by staff, sorted by sales total in descending order. */
67
68 • SELECT S.SALESID, ST.STAFFNAME,S.SALESTOTAL
69 FROM SALES S, STAFF ST
70 WHERE S.STAFFID = ST.STAFFID
71 ORDER BY S.SALESTOTAL DESC;
```

The result grid displays the following data:

SALESID	STAFFNAME	SALESTOTAL
SAL019	AHMAD SYAFIQ BIN YUSOF	19.50
SAL021	DANGA BIODNA BENITI FORDAUS	19.00
SAL015	AHMAD SYAFIQ BIN YUSOF	11.00
SAL002	AHMAD SYAFIQ BIN YUSOF	10.00
SAL011	AHMAD SYAFIQ BIN YUSOF	10.00
SAL018	DANGA BIODNA BENITI FORDAUS	9.50
SAL009	AHMAD SYAFIQ BIN YUSOF	8.50
SAL003	DANGA BIODNA BENITI FORDAUS	8.50
SAL014	DANGA BIODNA BENITI FORDAUS	8.50
SAL017	AHMAD SYAFIQ BIN YUSOF	8.00
SAL007	AHMAD SYAFIQ BIN YUSOF	7.50
SAL023	DANGA BIODNA BENITI FORDAUS	6.50
SAL006	DANGA BIODNA BENITI FORDAUS	6.00
SAL005	AHMAD SYAFIQ BIN YUSOF	5.50
SAL004	DANGA BIODNA BENITI FORDAUS	5.50
SAL001	AHMAD SYAFIQ BIN YUSOF	5.00
SAL008	DANGA BIODNA BENITI FORDAUS	5.00
SAL013	AHMAD SYAFIQ BIN YUSOF	4.50
SAL012	DANGA BIODNA BENITI FORDAUS	4.50
SAL010	DANGA BIODNA BENITI FORDAUS	3.00
SAL016	DANGA BIODNA BENITI FORDAUS	2.50

- i. 1 SQL statement to list significant information from a single table that is using aggregate function. (Aggregate Function)

The screenshot shows a SQL IDE with a query window and a result grid. The query is as follows:

```
75 /* QUERY: To display the total sales amount for each payment method on 6 January 2026
76 in order to analyse customer payment preferences and support the cashier's end-of-day closing report. */
77
78 • SELECT SALESPAYMETHOD, SUM(SALESTOTAL) AS TOTAL_SALES, SALESDATE
79 FROM SALES
80 WHERE SALESDATE = '2026-01-06'
81 GROUP BY SALESPAYMETHOD, SALESDATE;
```

The result grid displays the following data:

SALESPAYMETHOD	TOTAL_SALES	SALESDATE
E-WALLET	26.00	2026-01-06
CASH	19.00	2026-01-06

- j. 1 SQL statement to list significant information from a single table that is using aggregate function and GROUP BY clause. (Aggregate Function)

The screenshot shows a SQL IDE with a query window and a result grid. The query is designed to calculate the total number of items sold and the total sales value for each sales date, grouped by date.

```
86 /* QUERY: To display the total number of items sold and total sales value for each sales
87    date in order to generate daily sales summaries. */
88
89 * SELECT S.SALESDATE,SUM(OM.QUANTITY) AS "TOTAL ITEMS",SUM(DISTINCT S.SALESTOTAL) AS "DAILY_TOTAL"
90 FROM SALES S,ORDERS O , ORDERMENU OM
91 WHERE S.ORDERID = O.ORDERID
92 AND O.ORDERID = OM.ORDERID
93 GROUP BY S.SALESDATE;
94
```

The result grid displays the following data:

SALESDATE	TOTAL ITEMS	DAILY_TOTAL
2026-01-03	14	29.00
2026-01-04	8	30.00
2026-01-05	16	34.00
2026-01-06	11	45.00

- k. 1 SQL statement to list significant information from many tables that have relationship using aggregate function and GROUP BY clause. (Aggregate Function + Join)

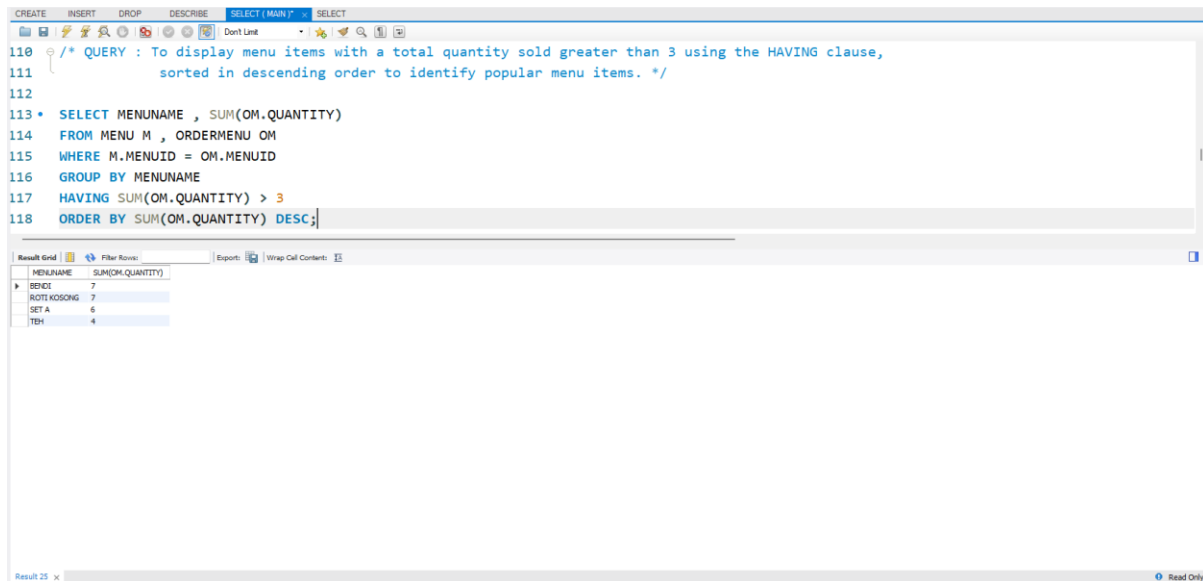
The screenshot shows a SQL IDE with a query window and a result grid. The query is designed to calculate the total quantity sold and the total sales value for each menu item, grouped by menu name and category, and ordered by total sales in descending order.

```
97 /* QUERY : To display the total quantity sold and total sales value for each menu item grouped
98    by menu name and category, ordered by total sales in descending order. */
99
100 * SELECT MENUNAME , MENCATEGORY ,SUM(OM.QUANTITY) AS "TOTAL QUANTITY", SUM(SUBTOTAL) AS "TOTAL SALES"
101 FROM MENU M , ORDERS O , ORDERMENU OM
102 WHERE M.MENUID = OM.MENUID
103 AND O.ORDERID = OM.ORDERID
104 GROUP BY MENUNAME,MENCATEGORY
105 ORDER BY SUM(SUBTOTAL) DESC ;
```

The result grid displays the following data:

MENUNAME	MENCATEGORY	TOTAL QUANTITY	TOTAL SALES
SET A	SET	6	51.00
SET B	SET	2	15.00
KARI KAMISING	LAKUK	2	14.00
SET C	SET	2	13.00
SOTONG	LAKUK	2	12.00
ROTI KOSONG	ROTI CANAI	7	10.50
TEH	MENJAMAN	4	9.00
AYAM GORENG	LAKUK	2	8.00
ROTI KAHWIN	ROTI CANAI	2	8.00
BANDUNG	MENJAMAN	3	7.50
MLO	MENJAMAN	2	7.00
IKAN MERAH	LAKUK	1	6.00
ROTI TELUR	ROTI CANAI	2	5.00
ROTI BOM	ROTI CANAI	2	5.00
AYAM KARI	LAKUK	1	4.00
AYAM KICAP	LAKUK	1	4.00
ROTI SARDIN	ROTI CANAI	1	4.00
ROTI MLO	ROTI CANAI	1	3.00
KOPIS	MENJAMAN	1	2.50
BEKUT	ADD-ON	7	2.00
PAPADOM	ADD-ON	1	1.50
TEMPPE GORENG	ADD-ON	1	0.50

- l. 1 SQL statement to list significant information from many tables that have relationship using aggregate function, GROUP BY and HAVING clause. (Aggregate function + Join)

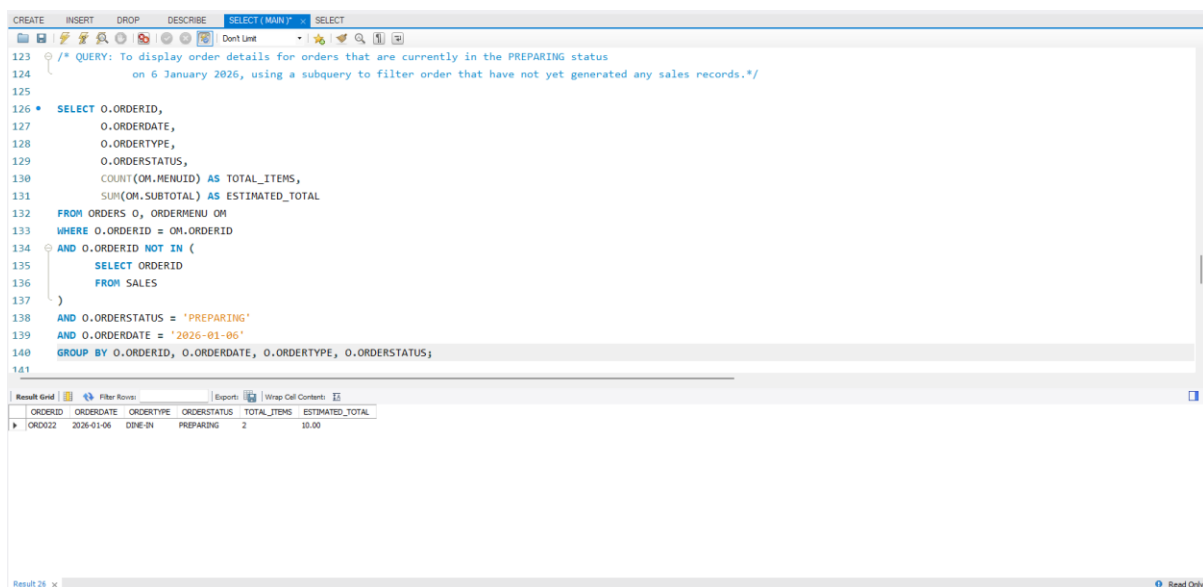


The screenshot shows a SQL query in a database client. The query is designed to list menu items with a total quantity sold greater than 3, sorted in descending order. The query uses a JOIN between the MENU and ORDERMENU tables, followed by a GROUP BY on MENUITEM and a HAVING clause to filter items with a sum of quantity greater than 3. The results are displayed in a grid with two columns: MENUITEM and SUM(OM.QUANTITY).

```
110 /* QUERY : To display menu items with a total quantity sold greater than 3 using the HAVING clause,
111 sorted in descending order to identify popular menu items. */
112
113 * SELECT MENUITEM , SUM(OM.QUANTITY)
114 FROM MENU M , ORDERMENU OM
115 WHERE M.MENUID = OM.MENUID
116 GROUP BY MENUITEM
117 HAVING SUM(OM.QUANTITY) > 3
118 ORDER BY SUM(OM.QUANTITY) DESC;
```

MENUITEM	SUM(OM.QUANTITY)
BECK	7
ROTHKOSONG	7
SET A	6
TDH	4

- m. 1 SQL statement to list significant information from a single or many tables using subqueries. (Subqueries)

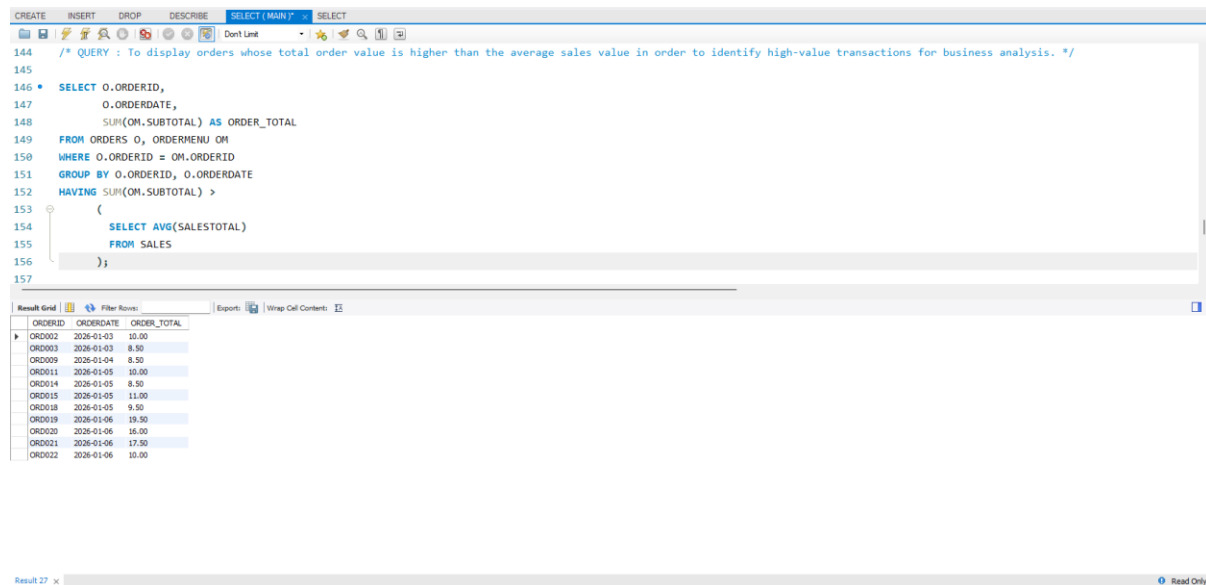


The screenshot shows a SQL query in a database client. The query is designed to display order details for orders that are currently in the PREPARING status on 6 January 2026, using a subquery to filter orders that have not yet generated any sales records. The query selects order details from the ORDERS and ORDERMENU tables, excluding orders that are in the PREPARING status and have a date of 2026-01-06. The results are displayed in a grid with columns: ORDERID, ORDERDATE, ORDERTYPE, ORDERSTATUS, TOTAL_ITEMS, and ESTIMATED_TOTAL.

```
123 /* QUERY: To display order details for orders that are currently in the PREPARING status
124 on 6 January 2026, using a subquery to filter order that have not yet generated any sales records.*/
125
126 * SELECT O.ORDERID,
127 O.ORDERDATE,
128 O.ORDERTYPE,
129 O.ORDERSTATUS,
130 COUNT(OM.MENUID) AS TOTAL_ITEMS,
131 SUM(OM.SUBTOTAL) AS ESTIMATED_TOTAL
132 FROM ORDERS O, ORDERMENU OM
133 WHERE O.ORDERID = OM.ORDERID
134 AND O.ORDERID NOT IN (
135 SELECT ORDERID
136 FROM SALES
137 )
138 AND O.ORDERSTATUS = 'PREPARING'
139 AND O.ORDERDATE = '2026-01-06'
140 GROUP BY O.ORDERID, O.ORDERDATE, O.ORDERTYPE, O.ORDERSTATUS;
```

ORDERID	ORDERDATE	ORDERTYPE	ORDERSTATUS	TOTAL_ITEMS	ESTIMATED_TOTAL
ORD022	2026-01-06	DINE-IN	PREPARING	2	10.00

- n. 1 SQL statement to list significant information from a single or many tables using subqueries.
(Subqueries)



The screenshot shows a SQL IDE interface with a query editor and a results grid. The query is designed to find orders with a total value higher than the average sales value.

```
144 /* QUERY : To display orders whose total order value is higher than the average sales value in order to identify high-value transactions for business analysis. */
145
146 * SELECT O.ORDERID,
147        O.ORDERDATE,
148        SUM(OH.SUBTOTAL) AS ORDER_TOTAL
149 FROM ORDERS O, ORDERMENU OM
150 WHERE O.ORDERID = OM.ORDERID
151 GROUP BY O.ORDERID, O.ORDERDATE
152 HAVING SUM(OH.SUBTOTAL) >
153 (
154     SELECT AVG(SALESTOTAL)
155     FROM SALES
156 );
157
```

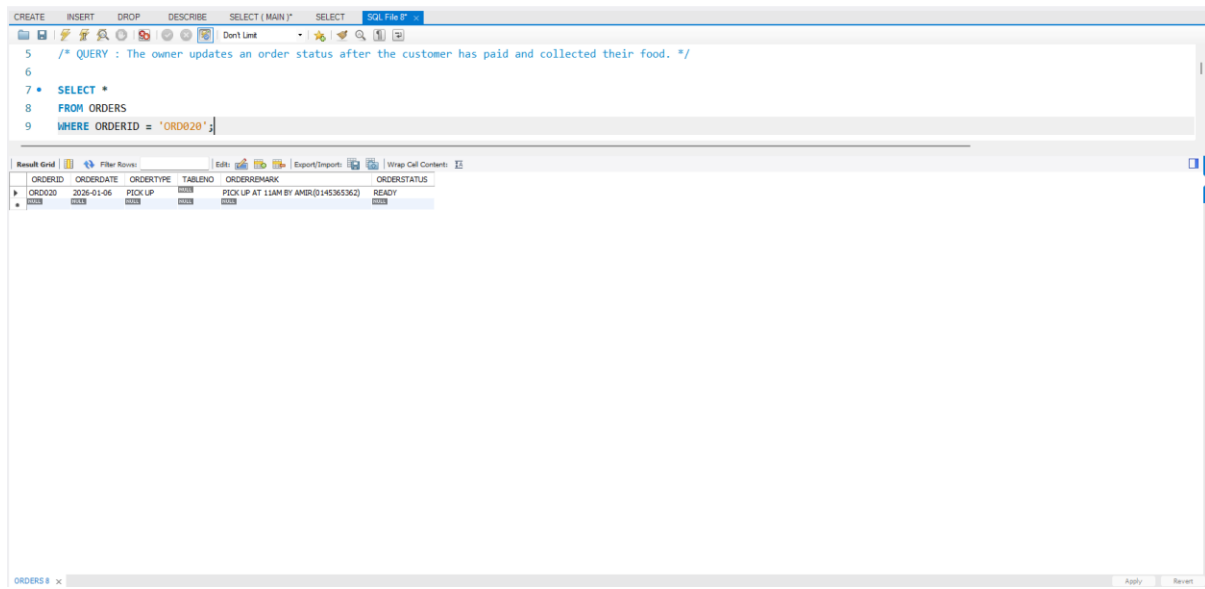
The results grid displays the following data:

ORDERID	ORDERDATE	ORDER_TOTAL
ORD002	2026-01-03	10.00
ORD003	2026-01-03	8.50
ORD009	2026-01-04	8.50
ORD011	2026-01-05	10.00
ORD014	2026-01-05	8.50
ORD015	2026-01-05	11.00
ORD018	2026-01-05	9.50
ORD019	2026-01-06	19.50
ORD020	2026-01-06	16.00
ORD021	2026-01-06	17.50
ORD022	2026-01-06	10.00

DML – UPDATE & DELETE

- o. Choose 1 table to perform UPDATE statement. Screenshot the statement and its output result. Then, ensure you also screenshot the SELECT * statement and its output result to see the changes.

BEFORE UPDATE :



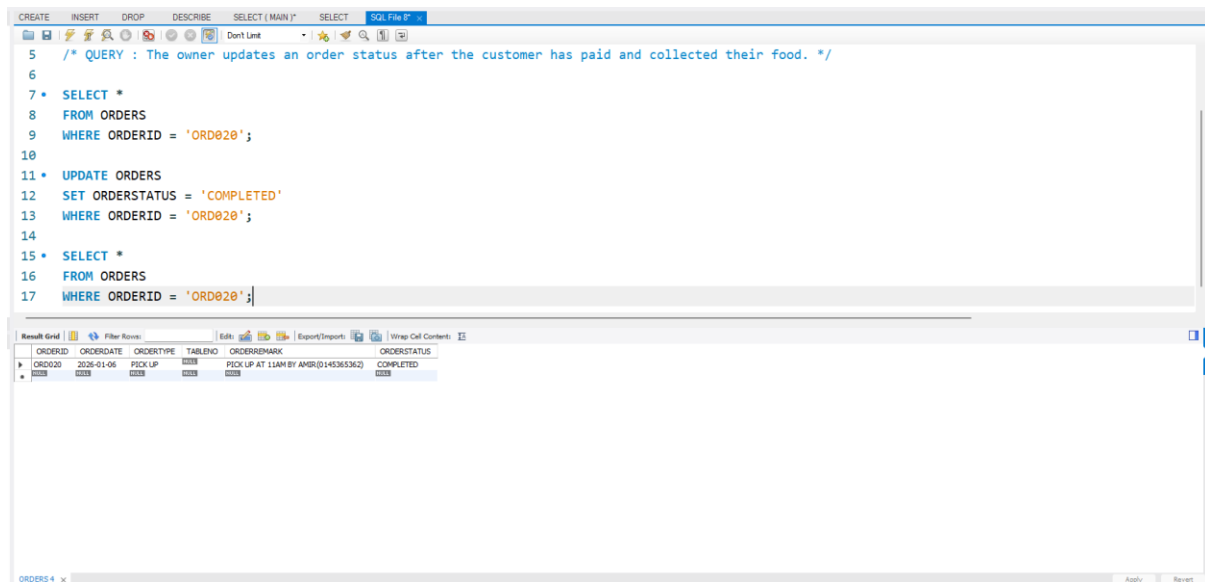
The screenshot shows the SQL Developer interface with a query window containing the following SQL statement:

```
5 /* QUERY : The owner updates an order status after the customer has paid and collected their food. */
6
7 * SELECT *
8 FROM ORDERS
9 WHERE ORDERID = 'ORD020';
```

Below the query window, the 'Result Grid' displays the data for the 'ORDERS' table:

ORDERID	ORDERDATE	ORDERTYPE	TABLERNO	ORDERREMARK	ORDERSTATUS
ORD020	2026-01-06	PICK UP	0000	PICK UP AT 11AM BY AMR(0145365362)	READY

AFTER UPDATE :



The screenshot shows the SQL Developer interface with a query window containing the following SQL statements:

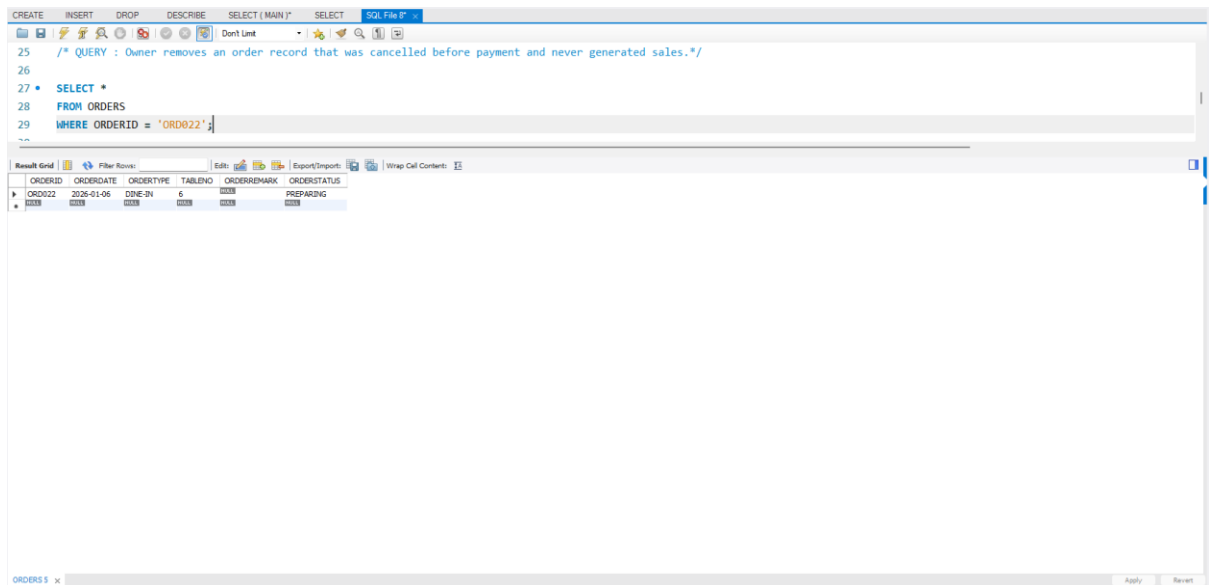
```
5 /* QUERY : The owner updates an order status after the customer has paid and collected their food. */
6
7 * SELECT *
8 FROM ORDERS
9 WHERE ORDERID = 'ORD020';
10
11 * UPDATE ORDERS
12 SET ORDERSTATUS = 'COMPLETED'
13 WHERE ORDERID = 'ORD020';
14
15 * SELECT *
16 FROM ORDERS
17 WHERE ORDERID = 'ORD020';
```

Below the query window, the 'Result Grid' displays the data for the 'ORDERS' table after the update:

ORDERID	ORDERDATE	ORDERTYPE	TABLERNO	ORDERREMARK	ORDERSTATUS
ORD020	2026-01-06	PICK UP	0000	PICK UP AT 11AM BY AMR(0145365362)	COMPLETED

- p. Choose 1 table to perform DELETE statement. Screenshot the statement and its output result. Then, ensure you also screenshot the SELECT * statement and its output result to see the changes.

BEFORE DELETE :



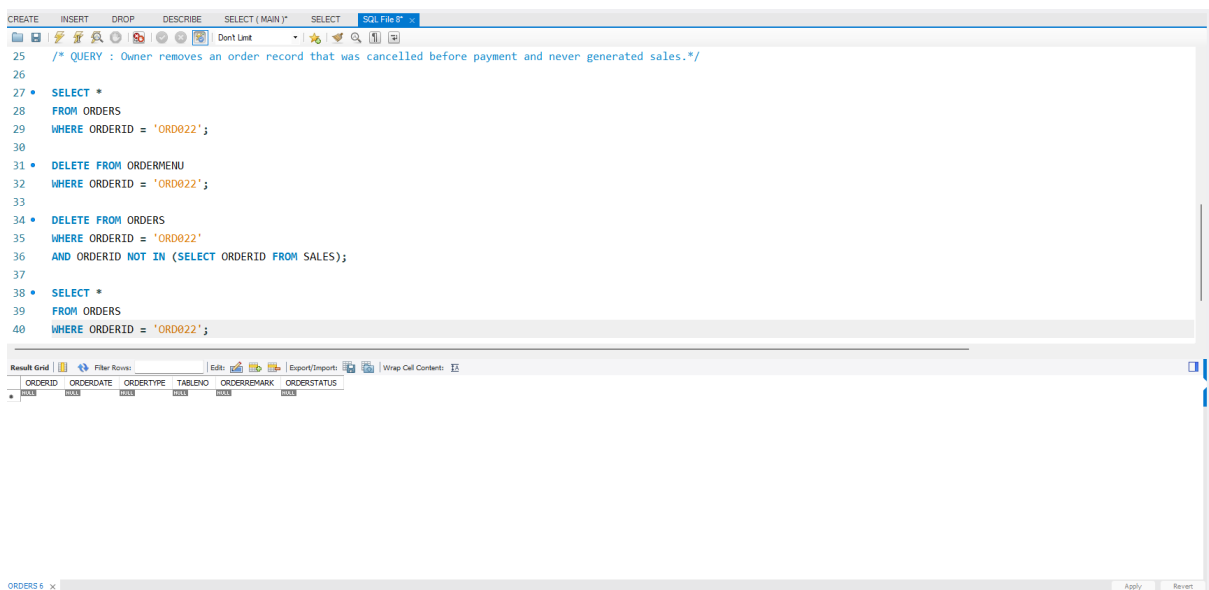
The screenshot shows the SQL Developer interface with a query window containing the following SQL statement:

```
25 /* QUERY : Owner removes an order record that was cancelled before payment and never generated sales.*/
26
27 • SELECT *
28 FROM ORDERS
29 WHERE ORDERID = 'ORD022';
```

The Results Grid below the query window displays the following data:

ORDERID	ORDERDATE	ORDERTYPE	TABLEN0	ORDERREMARK	ORDERSTATUS
ORD022	2025-01-06	DNE-DN	5		PREPARING

AFTER DELETE :



The screenshot shows the SQL Developer interface with a query window containing the following SQL statements:

```
25 /* QUERY : Owner removes an order record that was cancelled before payment and never generated sales.*/
26
27 • SELECT *
28 FROM ORDERS
29 WHERE ORDERID = 'ORD022';
30
31 • DELETE FROM ORDERMENU
32 WHERE ORDERID = 'ORD022';
33
34 • DELETE FROM ORDERS
35 WHERE ORDERID = 'ORD022'
36 AND ORDERID NOT IN (SELECT ORDERID FROM SALES);
37
38 • SELECT *
39 FROM ORDERS
40 WHERE ORDERID = 'ORD022';
```

The Results Grid below the query window displays the following data:

ORDERID	ORDERDATE	ORDERTYPE	TABLEN0	ORDERREMARK	ORDERSTATUS
---------	-----------	-----------	---------	-------------	-------------

5.0 REFERENCES

- Alfarisi, A. H. (2021). Dining house information system (Case study: Tengku Di Ujung Restaurant). *International Journal of Software Engineering and Computer Science*, 1(1), 13–20. <https://journal.lembagakita.org/index.php/ijsecs/article/view/323>
- Alfarisi, A. H. (2021). Inventory and orders information system in restaurant. *International Journal of Software Engineering and Computer Science*. <https://journal.lembagakita.org/ijsecs/article/view/323>
- B, V., Jothish, B., Pavithran, M., & Rajesh, R. (2023). Smart dining and smart food ordering system. *International Journal for Research in Applied Science and Engineering Technology*. <https://www.ijraset.com/best-journal/smart-food-ordering-method-and-system-for-dinning-889>
- Database Schema Design Guide for Restaurant Management Systems. *CodingEasyPeasy* (Online modeling examples). Access through <https://codingeasypeasy.com/blog/restaurant-management-system-database-design-schema-erd-and-examples/>
- GeeksforGeeks. (n.d.). How to design a database for online restaurant reservation and food delivery. <https://www.geeksforgeeks.org/dbms/how-to-design-a-database-for-online-restaurant-reservation-and-food-delivery/>
- Li, Y., Feng, A. X., Li, J., Mumick, S., Halevy, A., & Tan, W.-C. (2019). Subjective databases: Modeling experiential data (e.g., restaurant reviews). arXiv. <https://arxiv.org/abs/1902.09661>
- Inettutor. (n.d.). Online food ordering system ER diagram tutorial. <https://www.inettutor.com/diagrams/online-food-ordering-system-er-diagram/>
- Prabowo, W. A., & Wiguna, C. (2021). Designing of restaurant information system using rapid application development. *Sisforma Journal*. <https://journal.unika.ac.id/index.php/sisforma/article/view/3021>
- Rane, Y., Patil, S., & Salunkhe, S. (2022). Online food ordering system. *International Journal for Research in Applied Science and Engineering Technology*. <https://www.ijraset.com/best-journal/online-food-ordering-system>
- Restaurant Management System*. (2023). *Ijraset Journal for Research in Applied Science and Engineering Technology*. DOI: <https://www.ijraset.com/best-journal/restaurant-management-system>

Salakhm, P., & Talasee, J. (2024). Online restaurant management system: A case study of Redbox Restaurant. *Journal of Engineering and Industrial Technology*, 2(2), 48–60. https://doi.nrct.go.th/ListDoi/listDetail?Resolve_DOI=10.14456/jeit.2024.10

Salakhm, P., & Talasee, J. (2024). Restaurant database implementation using PHP and MySQL. *Journal of Engineering and Industrial Technology*. <https://ph03.tci-thaijo.org/index.php/JEIT/article/view/2212>

Salakhm, P., & Talasee, J. (2024). System Development Lifecycle usage for restaurant management system design. (Extracted from case study) Access through ph03.tci-thaijo.org

Talukder, A. A. T., Islam, M. A. I., Sultana, A., Pranto, T. H., & Rahman, R. M. (2022). A customer-centric food delivery system using blockchain and smart contracts. *arXiv*. <https://arxiv.org/abs/2209.04614>

University Journal of Student Projects and Research. (2024). Food Order Supporting System for Restaurant (ER Diagram example) — discusses ERD entities and tables. (PDF) Access through <https://ucsh.edu.mm/wp-content/uploads/2024/10/FOOD-ORDER-SUPPORTING-SYSTEM-FOR-RESTAURANT-2.pdf>

Warlina, L., & Noersidik, S. M. (2018). Designing web-based food ordering information system in restaurant. *IOP Conference Series: Materials Science and Engineering*, 407(1), 012029. <https://doi.org/10.1088/1757-899X/407/1/012029>

Warlina, L., & Noersidik, S. M. (2018). Designing Web-based Food Ordering Information System in Restaurant. *IOP Conference Series: Materials Science and Engineering*, 407(1), 012029. DOI:10.1088/1757-899X/407/1/012029 Access through https://www.researchgate.net/publication/327897483_Designing_Web-based_Food_Ordering_Information_System_in_Restaurant

Xiao Ting, M., & Alduais, N. (2024). Design and development of food ordering and management system for One Six Eight Restaurant. *Applied Information Technology and Computer Science*, 5(1), 493–512. <https://publisher.uthm.edu.my/periodicals/index.php/aitcs/article/view/12201>

Xie, Z. (2024). ER Diagram & MySQL Implementation for Restaurant Database. (Discussed in article) Access through https://www.researchgate.net/publication/378435479_Restaurant_management_database_system_design_and_implement

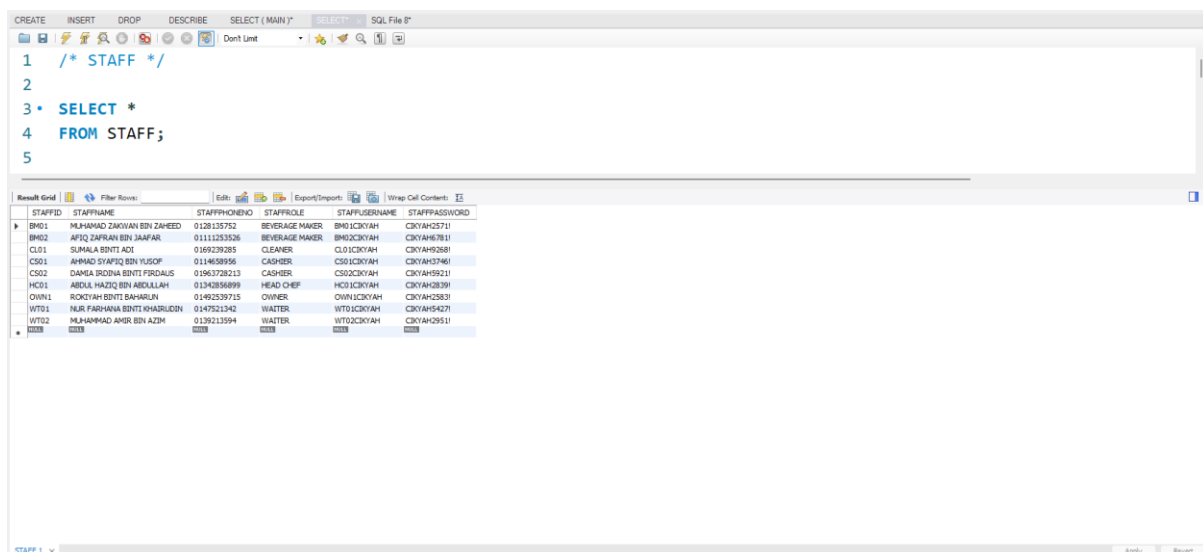
Xie, Z. (2024). Restaurant management database system: Design and implementation. *Applied and Computational Engineering*, 38(1), 210–218.
https://www.researchgate.net/publication/378435479_Restaurant_management_database_system_design_and_implementation

6.0 APPENDIX

6.1 Appendix A

For each table, copy and paste the INSERT record statement and screenshot
SELECT* statement and its output result.

```
/* ===== STAFF TABLE ===== */  
  
INSERT INTO STAFF VALUES ('OWN1','ROKIYAH BINTI  
BAHARUN','01492539715','OWNER','OWN1CIKYAH','CIKYAH2583!');  
  
INSERT INTO STAFF VALUES ('HC01','ABDUL HAZIQ BIN  
ABDULLAH','01342856899','HEAD CHEF','HC01CIKYAH','CIKYAH2839!');  
  
INSERT INTO STAFF VALUES ('BM01','MUHAMAD ZAKWAN BIN  
ZAHEED','0128135752','BEVERAGE MAKER','BM01CIKYAH','CIKYAH2571!');  
  
INSERT INTO STAFF VALUES ('BM02','AFIQ ZAFRAN BIN  
JAAFAR','01111253526','BEVERAGE MAKER','BM02CIKYAH','CIKYAH6781!');  
  
INSERT INTO STAFF VALUES ('CS01','AHMAD SYAFIQ BIN  
YUSOF','0114658956','CASHIER','CS01CIKYAH','CIKYAH3746!');  
  
INSERT INTO STAFF VALUES ('CS02','DAMIA IRDINA BINTI  
FIRDAUS','01963728213','CASHIER','CS02CIKYAH','CIKYAH5921!');  
  
INSERT INTO STAFF VALUES ('WT01','NUR FARHANA BINTI  
KHAIRUDIN','0147521342','WAITER','WT01CIKYAH','CIKYAH5427!');  
  
INSERT INTO STAFF VALUES ('WT02','MUHAMMAD AMIR BIN  
AZIM','0139213594','WAITER','WT02CIKYAH','CIKYAH2951!');  
  
INSERT INTO STAFF VALUES ('CL01','SUMALA BINTI  
ADI','0169239285','CLEANER','CL01CIKYAH','CIKYAH9268!');
```



STAFFID	STAFFNAME	STAFFPHONE	STAFFROLE	STAFFUSERNAME	STAFFPASSWORD
BM01	MUHAMAD ZAKWAN BIN ZAHEED	0128135752	BEVERAGE MAKER	BM01CIKYAH	CIKYAH2571
BM02	AFIQ ZAFRAN BIN JAAFAR	01111253526	BEVERAGE MAKER	BM02CIKYAH	CIKYAH6781
CL01	SUMALA BINTI ADI	0169239285	CLEANER	CL01CIKYAH	CIKYAH9268
CS01	AHMAD SYAFIQ BIN YUSOF	0114658956	CASHIER	CS01CIKYAH	CIKYAH3746
CS02	DAMIA IRDINA BINTI FIRDAUS	01963728213	CASHIER	CS02CIKYAH	CIKYAH5921
HC01	ABDUL HAZIQ BIN ABDULLAH	01342856899	HEAD CHEF	HC01CIKYAH	CIKYAH2839
OWN1	ROKIYAH BINTI BAHARUN	01492539715	OWNER	OWN1CIKYAH	CIKYAH2583
WT01	NUR FARHANA BINTI KHAIRUDIN	0147521342	WAITER	WT01CIKYAH	CIKYAH5427
WT02	MUHAMMAD AMIR BIN AZIM	0139213594	WAITER	WT02CIKYAH	CIKYAH2951

```

/* ===== MENU TABLE ===== */

/* ----- ROTI CANAI 6 -----*/
INSERT INTO MENU VALUES ('R01','ROTI KOSONG','ROTI
CANAI',1.50,NULL);
INSERT INTO MENU VALUES ('R02','ROTI TELUR','ROTI CANAI',2.50,NULL);
INSERT INTO MENU VALUES ('R03','ROTI BOM','ROTI CANAI',2.50,NULL);
INSERT INTO MENU VALUES ('R04','ROTI KAHWIN','ROTI
CANAI',4.00,NULL);
INSERT INTO MENU VALUES ('R05','ROTI SARDIN','ROTI
CANAI',4.00,NULL);
INSERT INTO MENU VALUES ('R06','ROTI MILO','ROTI CANAI',3.00,NULL);

/* ----- LAUK-LAU 7 -----*/
INSERT INTO MENU VALUES ('L01','AYAM GORENG','LAUK',4.00,'SATU
PIRING');
INSERT INTO MENU VALUES ('L02','AYAM KARI','LAUK',4.00,'SATU
PIRING');
INSERT INTO MENU VALUES ('L03','AYAM KICAP','LAUK',4.00,'SATU
PIRING');
INSERT INTO MENU VALUES ('L04','IKAN MERAH','LAUK',6.00,'SAIZ
KECIL');
INSERT INTO MENU VALUES ('L05','IKAN GORENG','LAUK',3.00,'SEEKOR');
INSERT INTO MENU VALUES ('L06','SOTONG','LAUK',6.00,'SAIZ BESAR');
INSERT INTO MENU VALUES ('L07','KARI KAMBING','LAUK',7.00,'SATU
PIRING');

/* ----- ADD-ON 4 -----*/
INSERT INTO MENU VALUES ('A01','TELUR REBUS','ADD-ON',1.50,NULL);
INSERT INTO MENU VALUES ('A02','BENDI','ADD-ON',0.50,NULL);
INSERT INTO MENU VALUES ('A03','PAPADOM','ADD-ON',1.50,NULL);
INSERT INTO MENU VALUES ('A04','TEMPE GORENG','ADD-ON',0.50,NULL);

/* ----- SET 3-----*/
INSERT INTO MENU VALUES ('S01','SET
A','SET',8.50,'NASI+AYAM+SAYUR+TELUR MASIN');

```

```
INSERT INTO MENU VALUES ('S02','SET
B','SET',7.50,'NASI+AYAM+SAYUR');
```

```
INSERT INTO MENU VALUES ('S03','SET C','SET',6.50,'NASI+AYAM');
```

```
/* ----- MINUMAN 5 -----*/
```

```
INSERT INTO MENU VALUES ('M01','TEH','MINUMAN',2.00,'PANAS');
```

```
INSERT INTO MENU VALUES ('M02','TEH','MINUMAN',2.50,'SEJUK');
```

```
INSERT INTO MENU VALUES ('M03','KOPI','MINUMAN',2.50,'SEJUK');
```

```
INSERT INTO MENU VALUES ('M04','MILO','MINUMAN',3.50,'SEJUK');
```

```
INSERT INTO MENU VALUES ('M05','BANDUNG','MINUMAN',2.50,'SEJUK');
```

CREATE INSERT DROP DESCRIBE SELECT (MAIN) SELECT* SQL File B"

7 /* MENU */

8

9 SELECT *

10 FROM MENU;

Result Grid Filter Rows: Edit Export/Import Wrap Cell Contents

MENUID	MENUNAME	MENUCATEGORY	MENUPRICE	MENUDISC
A01	TELUR REBUS	ADD-ON	1.50	
A02	BENCI	ADD-ON	0.50	
A03	PAPADOM	ADD-ON	1.50	
A04	TEMPE GORENG	ADD-ON	0.50	
L01	AYAM GORENG	LAUK	4.00	SATU PIRING
L02	AYAM KANI	LAUK	4.00	SATU PIRING
L03	AYAM KICAP	LAUK	4.00	SATU PIRING
L04	IKAN MERAH	LAUK	6.00	SAIZ KECIL
L05	IKAN GORENG	LAUK	3.00	SEKOR
L06	SOTONG	LAUK	6.00	SAIZ BESAR
L07	KARSI KAMBING	LAUK	7.00	SATU PIRING
M01	TEH	MINUMAN	2.00	PANAS
M02	TEH	MINUMAN	2.50	SEJUK
M03	KOPI	MINUMAN	2.50	SEJUK
M04	MILO	MINUMAN	3.50	SEJUK
M05	BANDUNG	MINUMAN	2.50	SEJUK
R01	ROTI KOSONG	ROTI CANAI	1.50	
R02	ROTI TELUR	ROTI CANAI	2.50	
R03	ROTI BOM	ROTI CANAI	2.50	
R04	ROTI KAWEN	ROTI CANAI	4.00	
R05	ROTI SARONG	ROTI CANAI	4.00	
R06	ROTI MILO	ROTI CANAI	3.00	
S01	SET A	SET	8.50	NASI+AYAM+SAYUR+TELUR MASIN
S02	SET B	SET	7.50	NASI+AYAM+SAYUR
S03	SET C	SET	6.50	NASI+AYAM

MENU 2 » Apply Revert

```

/* ----- ORDERS -----*/

INSERT INTO ORDERS VALUES('ORD001','2026-01-03','DINE-
IN',1,NULL,'COMPLETED');

INSERT INTO ORDERS VALUES('ORD002','2026-01-03','DINE-
IN',2,NULL,'COMPLETED');

INSERT INTO ORDERS
VALUES('ORD003','2026-01-03','TAKE-AWAY',NULL,NULL,'COMPLETED');

INSERT INTO ORDERS VALUES('ORD004','2026-01-03','DINE-
IN',3,NULL,'COMPLETED');

INSERT INTO ORDERS VALUES('ORD005','2026-01-03','TAKE-
AWAY',NULL,NULL,'COMPLETED');


INSERT INTO ORDERS VALUES('ORD006','2026-01-04','DINE-
IN',4,NULL,'COMPLETED');

INSERT INTO ORDERS VALUES('ORD007','2026-01-04','DINE-
IN',5,NULL,'COMPLETED');

INSERT INTO ORDERS VALUES('ORD008','2026-01-04','TAKE-
AWAY',NULL,NULL,'COMPLETED');

INSERT INTO ORDERS VALUES('ORD009','2026-01-04','DINE-
IN',6,NULL,'COMPLETED');

INSERT INTO ORDERS
VALUES('ORD010','2026-01-04','TAKE-AWAY',NULL,NULL,'COMPLETED');


INSERT INTO ORDERS VALUES('ORD011','2026-01-05','DINE-
IN',7,NULL,'COMPLETED');

INSERT INTO ORDERS VALUES('ORD012','2026-01-05','DINE-
IN',8,NULL,'COMPLETED');

INSERT INTO ORDERS VALUES('ORD013','2026-01-05','TAKE-
AWAY',NULL,NULL,'COMPLETED');

INSERT INTO ORDERS VALUES('ORD014','2026-01-05','DINE-
IN',9,NULL,'COMPLETED');

INSERT INTO ORDERS VALUES('ORD015','2026-01-05','DINE-
IN',10,NULL,'COMPLETED');

INSERT INTO ORDERS VALUES('ORD016','2026-01-05','TAKE-
AWAY',NULL,NULL,'COMPLETED');

INSERT INTO ORDERS VALUES('ORD017','2026-01-05','DINE-
IN',11,NULL,'COMPLETED');

```



```
INSERT INTO ORDERS VALUES ('ORD018','2026-01-05','DINE-
IN',12,NULL,'COMPLETED');
```

```
INSERT INTO ORDERS VALUES ('ORD019','2026-01-06','DINE-
IN',4,NULL,'COMPLETED');
```

```
INSERT INTO ORDERS
```

```
VALUES ('ORD020','2026-01-06','PICK UP',NULL,'PICK UP AT 11AM BY
AMIR(0145365362) ','READY');
```

```
INSERT INTO ORDERS VALUES ('ORD021','2026-01-06','TAKE-
AWAY',NULL,NULL,'COMPLETED');
```

```
INSERT INTO ORDERS VALUES ('ORD022','2026-01-06','DINE-
IN',6,NULL,'PREPARING');
```

```
INSERT INTO ORDERS
```

```
VALUES ('ORD023','2026-01-06','TAKE-AWAY',NULL,NULL,'COMPLETED');
```

CREATE INSERT DROP DESCRIBE SELECT (MAIN) SELECT SQL File 8"

13 /* ORDERS */

14

15 * SELECT *

16 FROM ORDERS;

Result Grid Filter Rows: Edit Export/Import Wrap Cell Contents

ORDERID	ORDERDATE	ORDERTYPE	TABLENO	ORDERREMARK	ORDERSTATUS
ORD001	2026-01-03	DINE-IN	1		COMPLETED
ORD002	2026-01-03	DINE-IN	2		COMPLETED
ORD003	2026-01-03	TAKE-AWAY			COMPLETED
ORD004	2026-01-03	DINE-IN	3		COMPLETED
ORD005	2026-01-03	TAKE-AWAY			COMPLETED
ORD006	2026-01-04	DINE-IN	4		COMPLETED
ORD007	2026-01-04	DINE-IN	5		COMPLETED
ORD008	2026-01-04	TAKE-AWAY			COMPLETED
ORD009	2026-01-04	DINE-IN	6		COMPLETED
ORD010	2026-01-04	TAKE-AWAY			COMPLETED
ORD011	2026-01-05	DINE-IN	7		COMPLETED
ORD012	2026-01-05	DINE-IN	8		COMPLETED
ORD013	2026-01-05	TAKE-AWAY			COMPLETED
ORD014	2026-01-05	DINE-IN	9		COMPLETED
ORD015	2026-01-05	DINE-IN	10		COMPLETED
ORD016	2026-01-05	TAKE-AWAY			COMPLETED
ORD017	2026-01-05	DINE-IN	11		COMPLETED
ORD018	2026-01-05	DINE-IN	12		COMPLETED
ORD019	2026-01-06	DINE-IN	4		COMPLETED
ORD020	2026-01-06	PICK UP		PICK UP AT 11AM BY AMIR(0145365362)	READY
ORD021	2026-01-06	TAKE-AWAY			COMPLETED
ORD022	2026-01-06	DINE-IN	6		PREPARING
ORD023	2026-01-06	TAKE-AWAY			COMPLETED

ORDERS 4 x Apply Revert

```

/* ----- ORDERMENU ----- */
INSERT INTO ORDERMENU VALUES ('R01','ORD001',2,3.00,NULL);
INSERT INTO ORDERMENU VALUES ('M01','ORD001',1,2.00,NULL);

INSERT INTO ORDERMENU VALUES ('R02','ORD002',1,2.50,NULL);
INSERT INTO ORDERMENU VALUES ('L01','ORD002',1,4.00,NULL);
INSERT INTO ORDERMENU VALUES ('M04','ORD002',1,3.50,NULL);

INSERT INTO ORDERMENU VALUES ('S01','ORD003',1,8.50,'LAUK ASING');

INSERT INTO ORDERMENU VALUES ('R04','ORD004',1,4.00,NULL);
INSERT INTO ORDERMENU VALUES ('A02','ORD004',3,1.50,NULL);

INSERT INTO ORDERMENU VALUES ('R01','ORD005',2,3.00,NULL);
INSERT INTO ORDERMENU VALUES ('M05','ORD005',1,2.50,NULL);

INSERT INTO ORDERMENU VALUES ('L02','ORD006',1,4.00,'KUAH LEBIH');
INSERT INTO ORDERMENU VALUES ('M01','ORD006',1,2.00,NULL);

INSERT INTO ORDERMENU VALUES ('S02','ORD007',1,7.50,NULL);

INSERT INTO ORDERMENU VALUES ('R03','ORD008',2,5.00,'ROTI GARING');

INSERT INTO ORDERMENU VALUES ('L04','ORD009',1,6.00,NULL);
INSERT INTO ORDERMENU VALUES ('M02','ORD009',1,2.50,NULL);

INSERT INTO ORDERMENU VALUES ('R06','ORD010',1,3.00,NULL);

INSERT INTO ORDERMENU VALUES ('S03','ORD011',1,6.50,NULL);
INSERT INTO ORDERMENU VALUES ('M04','ORD011',1,3.50,'SUSU LEBIH');

INSERT INTO ORDERMENU VALUES ('R05','ORD012',1,4.00,NULL);

```

```

INSERT INTO ORDERMENU VALUES ('A04','ORD012',1,0.50,NULL);

INSERT INTO ORDERMENU VALUES ('R01','ORD013',3,4.50,NULL);

INSERT INTO ORDERMENU VALUES ('L06','ORD014',1,6.00,NULL);
INSERT INTO ORDERMENU VALUES ('M02','ORD014',1,2.50,NULL);

INSERT INTO ORDERMENU VALUES ('S01','ORD015',1,8.50,NULL);
INSERT INTO ORDERMENU VALUES ('M05','ORD015',1,2.50,NULL);

INSERT INTO ORDERMENU VALUES ('R02','ORD016',1,2.50,NULL);

INSERT INTO ORDERMENU VALUES ('R04','ORD017',1,4.00,NULL);
INSERT INTO ORDERMENU VALUES ('L03','ORD017',1,4.00,NULL);

INSERT INTO ORDERMENU VALUES ('L07','ORD018',1,7.00,NULL);
INSERT INTO ORDERMENU VALUES ('M03','ORD018',1,2.50,'KURANG MANIS');

INSERT INTO ORDERMENU VALUES ('S01','ORD019',2,17.00,NULL);
INSERT INTO ORDERMENU VALUES ('M05','ORD019',1,2.50,NULL);

INSERT INTO ORDERMENU VALUES ('S02','ORD020',1,7.50,NULL);
INSERT INTO ORDERMENU VALUES ('S01','ORD020',1,8.50,NULL);

INSERT INTO ORDERMENU VALUES ('L07','ORD021',1,7.00,NULL);
INSERT INTO ORDERMENU VALUES ('S01','ORD021',1,8.50,'KURANG MANIS');
INSERT INTO ORDERMENU VALUES ('A03','ORD021',1,1.50,NULL);
INSERT INTO ORDERMENU VALUES ('A02','ORD021',4,0.50,NULL);

INSERT INTO ORDERMENU VALUES ('L01','ORD022',1,4.00,NULL);
INSERT INTO ORDERMENU VALUES ('L06','ORD022',1,6.00,NULL);

```

INSERT INTO ORDERMENU VALUES ('S03','ORD023',1,6.50,NULL);

CREATE INSERT DROP DESCRIBE SELECT (MAIN) * SELECT * SQL File B*

19 /* ORDERMENU */

20

21 * SELECT *

22 FROM ORDERMENU;

Result Grid Filter Rows: Edit: Export/Import Wrap Cell Contents

	MENUID	ORDERID	QUANTITY	SUBTOTAL	REQUEST
A02	ORD004	3	1.50		
A02	ORD021	4	0.50		
A03	ORD021	1	1.50		
A04	ORD012	1	0.50		
L01	ORD002	1	4.00		
L01	ORD022	1	4.00		
L02	ORD006	1	4.00		KLAH LEBEH
L03	ORD017	1	4.00		
L04	ORD009	1	6.00		
L06	ORD014	1	6.00		
L06	ORD022	1	6.00		
L07	ORD018	1	7.00		
L07	ORD021	1	7.00		
M01	ORD001	1	2.00		
M01	ORD006	1	2.00		
M02	ORD009	1	2.50		
M02	ORD014	1	2.50		
M03	ORD018	1	2.50		KURANG MANES
M04	ORD002	1	3.50		
M04	ORD011	1	3.50		SUSU LEBEH
M05	ORD005	1	2.50		
M05	ORD015	1	2.50		
M05	ORD019	1	2.50		
R01	ORD001	2	3.00		
R01	ORD003	2	3.00		
R01	ORD013	3	4.50		
R02	ORD002	1	2.50		
R02	ORD016	1	2.50		
R03	ORD008	2	5.00		ROTT GARING
R04	ORD004	1	4.00		
R04	ORD017	1	4.00		
R06	ORD012	1	4.00		
R06	ORD010	1	3.00		

ORDERMENU 6 X

Apply Revert

```

/* ----- SALES -----*/

INSERT INTO SALES VALUES ('SAL001','ORD001','CS01','2026-01-
03',5.00,'CASH');

INSERT INTO SALES VALUES ('SAL002','ORD002','CS01','2026-01-
03',10.00,'E-WALLET');

INSERT INTO SALES VALUES ('SAL003','ORD003','CS02','2026-01-
03',8.50,'CASH');

INSERT INTO SALES VALUES ('SAL004','ORD004','CS02','2026-01-
03',5.50,'E-WALLET');

INSERT INTO SALES VALUES ('SAL005','ORD005','CS01','2026-01-
03',5.50,'CASH');


INSERT INTO SALES VALUES ('SAL006','ORD006','CS02','2026-01-
04',6.00,'CASH');

INSERT INTO SALES VALUES ('SAL007','ORD007','CS01','2026-01-
04',7.50,'E-WALLET');

INSERT INTO SALES VALUES ('SAL008','ORD008','CS02','2026-01-
04',5.00,'E-WALLET');

INSERT INTO SALES VALUES ('SAL009','ORD009','CS01','2026-01-
04',8.50,'CASH');

INSERT INTO SALES VALUES ('SAL010','ORD010','CS02','2026-01-
04',3.00,'CASH');


INSERT INTO SALES VALUES ('SAL011','ORD011','CS01','2026-01-
05',10.00,'E-WALLET');

INSERT INTO SALES VALUES ('SAL012','ORD012','CS02','2026-01-
05',4.50,'CASH');

INSERT INTO SALES VALUES ('SAL013','ORD013','CS01','2026-01-
05',4.50,'E-WALLET');

INSERT INTO SALES VALUES ('SAL014','ORD014','CS02','2026-01-
05',8.50,'E-WALLET');

INSERT INTO SALES VALUES ('SAL015','ORD015','CS01','2026-01-
05',11.00,'CASH');

INSERT INTO SALES VALUES ('SAL016','ORD016','CS02','2026-01-
05',2.50,'CASH');

INSERT INTO SALES VALUES ('SAL017','ORD017','CS01','2026-01-
05',8.00,'E-WALLET');

```

```
INSERT INTO SALES VALUES ('SAL018','ORD018','CS02','2026-01-05',9.50,'E-WALLET');
```

```
INSERT INTO SALES VALUES ('SAL019','ORD019','CS01','2026-01-06',19.50,'E-WALLET');
```

```
INSERT INTO SALES VALUES ('SAL021','ORD021','CS02','2026-01-06',19.00,'CASH');
```

```
INSERT INTO SALES VALUES ('SAL023','ORD023','CS02','2026-01-06',6.50,'E-WALLET');
```

```
COMMIT;
```

The screenshot shows a SQL IDE interface. The top toolbar includes buttons for CREATE, INSERT, DROP, DESCRIBE, SELECT (MAIN), and a SQL File icon. Below the toolbar, a query window contains the following SQL code:

```
25 /* SALES */
26
27 * SELECT *
28 FROM SALES;
29
```

Below the query window, a 'Result Grid' displays the results of the query. The grid has columns for SALESID, ORDERID, STAFFID, SALESDATE, SALESTOTAL, and SALES/PAID/METHOD. It contains 23 rows of data, with the last three rows (SAL021, SAL022, SAL023) corresponding to the INSERT statements in the previous blocks.

SALESID	ORDERID	STAFFID	SALESDATE	SALESTOTAL	SALES/PAID/METHOD
SAL001	ORD001	CS01	2026-01-03	5.00	CASH
SAL002	ORD002	CS01	2026-01-03	10.00	E-WALLET
SAL003	ORD003	CS02	2026-01-03	8.50	CASH
SAL004	ORD004	CS02	2026-01-03	5.50	E-WALLET
SAL005	ORD005	CS01	2026-01-03	5.50	CASH
SAL006	ORD006	CS02	2026-01-04	6.00	CASH
SAL007	ORD007	CS01	2026-01-04	7.50	E-WALLET
SAL008	ORD008	CS02	2026-01-04	5.00	E-WALLET
SAL009	ORD009	CS01	2026-01-04	8.50	CASH
SAL010	ORD010	CS02	2026-01-04	3.00	CASH
SAL011	ORD011	CS01	2026-01-05	10.00	E-WALLET
SAL012	ORD012	CS02	2026-01-05	4.50	CASH
SAL013	ORD013	CS01	2026-01-05	4.50	E-WALLET
SAL014	ORD014	CS02	2026-01-05	8.50	E-WALLET
SAL015	ORD015	CS01	2026-01-05	11.00	CASH
SAL016	ORD016	CS02	2026-01-05	2.50	CASH
SAL017	ORD017	CS01	2026-01-05	8.00	E-WALLET
SAL018	ORD018	CS02	2026-01-05	9.50	E-WALLET
SAL019	ORD019	CS01	2026-01-06	19.50	E-WALLET
SAL021	ORD021	CS02	2026-01-06	19.00	CASH
SAL022	ORD022	CS02	2026-01-06	6.50	E-WALLET
SAL023	ORD023	CS02	2026-01-06	6.50	E-WALLET

6.2 Appendix B

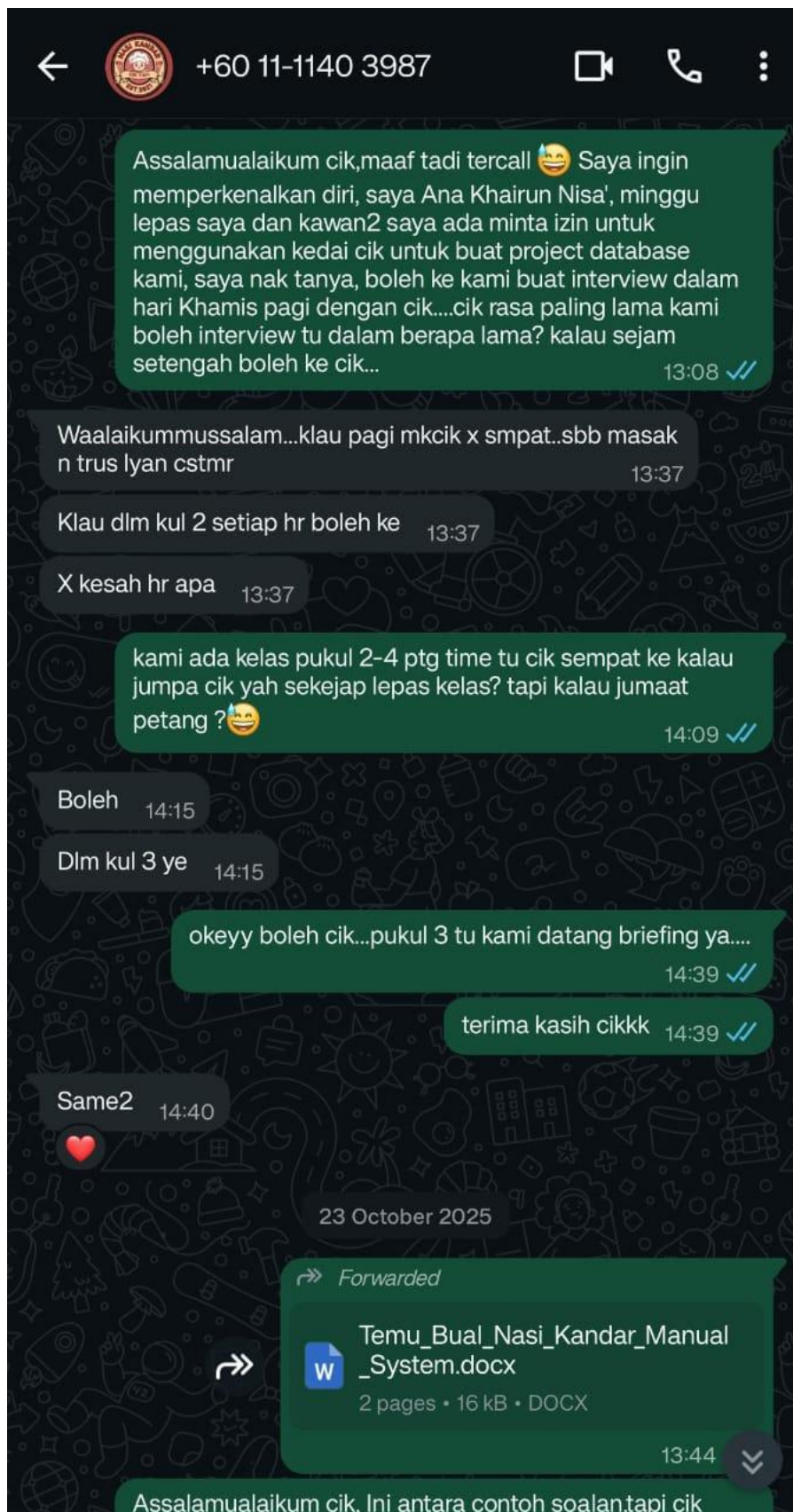


Figure 8 : Evidence 1 of Interview Session

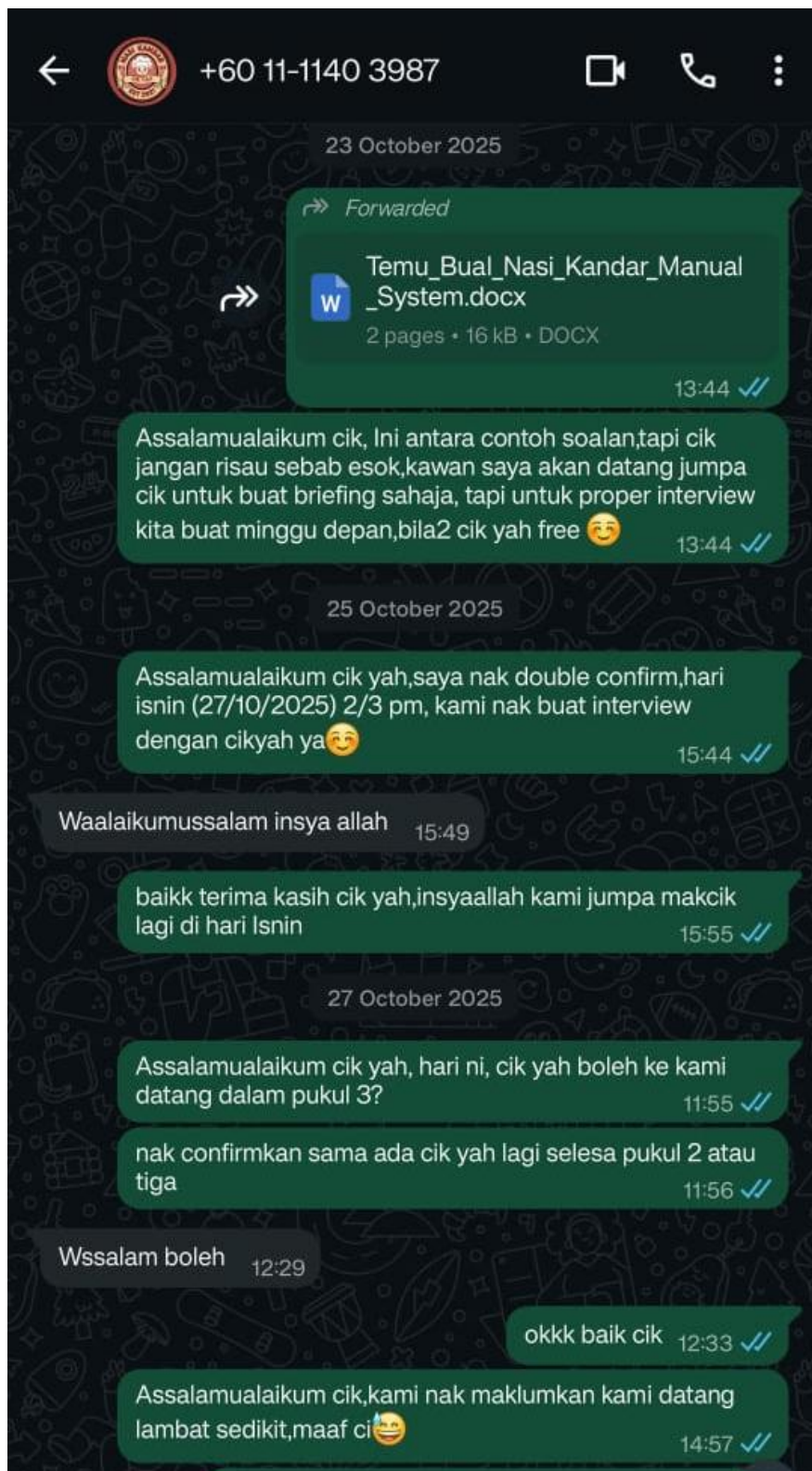


Figure 9 : Evidence 2 of Interview Session

INTERVIEW Q&A WITH NASI KANDAR BUSINESS



Figure 10 : Group Members with the Representative of the Company

SECTION A : COMPANY BACKGROUND

1. When did this Nasi Kandar business start operating?

A: I started my Nasi Kandar business in 2021 at the Gamma area. However, I have been involved in the food industry since 2007, when I first ran a small food business in Terengganu. In 2017, I moved to Tapah after being awarded a tender to operate within a university premise, and then I moved out to get my own place and have continued managing my Nasi Kandar restaurant here ever since.

2. What type of food or main services does this restaurant offer?

A: My restaurant mainly serves Nasi Kandar dishes and affordable set meals for students, such as Set A, Set B, and Set C. Each set includes rice, chicken, vegetables, eggs, and mixed gravy, with prices starting from RM6.50. Customers can also request additional side dishes like mutton curry, squid, fried fish, boiled eggs, or vegetables. Besides Nasi Kandar, I also serve *roti canai* and *nasi lemak* for breakfast.

3. How many employees work here and what are their main tasks?

A: I have several full-time workers, including some of my family members who help with daily operations. Each person has specific duties, some cook, others prepare beverages, take customer orders, clean tables, and buy raw ingredients. My husband is responsible for purchasing daily supplies, while I manage the kitchen and personally prepare *roti canai* for customers.

4. What is the restaurant's daily operating hours?

A: We operate daily from 7:30 a.m. until 4:00 p.m. Previously, the restaurant was open until 11:00 p.m., but I decided to shorten the hours due to staff limitations and workload. Currently, we operate continuously without a formal rest break.

SECTION B : DAILY OPERATIONS

5. How are customer orders taken at this restaurant?

A: Customer orders are taken manually. For rice dishes, customers queue and choose their preferred items, while for beverages and *roti canai*, the staff or I write down the orders on paper slips before preparing them.

6. How are these orders recorded — by book, order slip, or receipt?

A: We record all orders by hand using paper order slips. At the moment, I do not use any digital or automated system for order recording.

7. Who is responsible for taking and recording customer orders?

A: Either I take the orders myself or one of my family members helps, depending on who is available. During busy hours, everyone helps out to ensure orders are taken and served efficiently.

8. How is customer payment calculated and recorded each day?

A: All payments are calculated manually using a calculator. Most customers pay either in cash or via online bank transfer. I usually check my bank account after each transaction to confirm payments received.

9. Does the restaurant prepare any daily or monthly sales records?

A: Yes, I record daily sales manually in a record book. I used to prepare monthly summaries as well, but due to time constraints, I now focus mainly on keeping daily records up to date.

SECTION C : MANUAL DATA MANAGEMENT

10. Where are the sales, order, and stock records kept?

A: All sales, order, and stock records are written and kept in record books. These serve as the main documentation for my business transactions and inventory tracking.

11. Who manages and keeps these files or record books?

A: I personally manage and update all the records myself. I prefer to handle them directly to ensure accuracy and consistency.

12. How frequently are these records updated (daily, weekly, or monthly)?

A: I update the records every day after each transaction or purchase. This helps me keep the information current and reliable.

13. How does the restaurant retrieve old information when needed?

A: Retrieving old information is quite easy since each record is dated by day, month, and year. I can look back at previous entries whenever necessary.

14. Has there ever been a situation where files or records were lost or damaged?

A: Thankfully, I have never lost any records. However, sometimes the record books get wet or stained with oil during cooking, which can make the writing slightly faded, but still readable.

15. How is daily income calculated, like manually or using a calculator?

A: I calculate daily income manually with a calculator to minimize mistakes and ensure accuracy.

SECTION D : PROBLEMS AND CHALLENGES

16. What are the main problems faced when using a manual system?

A: The main problems I face are miscalculations and having to recheck the figures several times to make sure everything is correct. Occasionally, some sales or expenses are forgotten and not recorded, causing small inconsistencies in my financial records.

17. Does searching for old records take a long time?

A: Not usually. Since I organize all records by date, I can find recent information quickly. However, looking for older entries can still be a bit time-consuming.

18. Have there been any miscalculations or confusion between customer orders?

A: Yes, such mistakes happen occasionally, especially when the restaurant is very busy. Sometimes an order is written incorrectly, or a food item is missed, leading to confusion between tables.

19. How is the operation during peak hours, does the manual system make it difficult?

A: During peak lunch hours, the restaurant becomes crowded, and the manual order-taking process becomes more difficult. My staff and I often need to multitask for example serving customers while writing orders — which slows down our work and increases the chance of errors.

20. Has there ever been a delay or mistake in orders due to the manual system?

A: Yes, delays and mistakes do occur at times. For example, if the table number is not written on the order slip, it causes confusion in delivering the right food to the right table.

SECTION E : SUGGESTIONS AND EXPECTATIONS FOR A COMPUTERIZED SYSTEM

21. If given the chance to use a computerized system, what features or functions would you like it to have?

A: I would like a system that can automatically record orders, payments, and receipts so that all information is consistent and well-organized. This would also help reduce confusion and save time in managing daily operations.

22. Do you believe that using a computerized system could simplify and speed up daily operations?

A: Yes, I strongly believe a computerized system would make daily tasks more efficient, speed up order-taking, and reduce human errors. It would also help keep all records accurate and easy to review.

23. How do the workers in your shop feel about using computers or tablets for taking orders?

A: I believe my workers, especially the younger ones, would adapt quickly to using computers or tablets. Some brief training might be needed, but I am confident they would find it easy to learn.

24. What are your expectations if a database system is implemented (e.g., automatic sales reports, customer records, or stock management)?

A: I expect that a computerized database system would greatly simplify my business management. It could automatically generate daily and monthly reports, track sales, monitor stock, and manage expenses efficiently. This would reduce my workload and improve overall accuracy in managing the restaurant.

RUBRIC FORM

Before printing, fill in the table below:

SEMESTER	20254
CLASS GROUP (3A / 3B / 3C / 3D / 3E / 3F)	3F
GROUP NAME	Minion
GROUP MEMBERS	
	1. Nurul Aiesyah Fatihah binti Norazehasram (0125352178)
	2. Ana Khairun Nisa' binti Mohd Zaidi
	3. Shannah Love Olas
	4. Norsyarida binti Rosli
NAME OF DATABASE SYSTEM	Food Ordering Database System
COMPANY NAME	Nasi Kandar Cik Yah

<i>Presentation:</i>	<i>Report:</i>	TOTAL:
		116

GROUP PROJECT RUBRIC

A) PROJECT REPORT (76 marks)

i) Database Requirements Rubric

Criteria	Score					Weight	Total / Section	MARK GIVEN
	1 (POOR)	2 (BELOW AVERAGE)	3 (AVERAGE)	4 (GOOD)	5 (EXCELLENT)	w		
Report Format	Report poorly formatted; inconsistent layout	Some formatting inconsistencies; lacks clarity	Adequately formatted. Minor layout issues	Well formatted with clear headings and spacing	Professional, consistent and polished formatting	0.6	3	
Company Background	Missing or irrelevant company information	Superficial or confusing description	Basic background info present and understandable	Clear and concise with relevant company details	Comprehensive, Insightful background with context	1	5	
Daily Operation Description	Operations poorly described or missing	Overview of operations unclear or incomplete	Basic description of operations	Clear description with sufficient detail	Thorough, detailed and well-structured explanation. Supported with diagram	1	5	
Problem Findings	Problem statement is missing or vague. No specific data management problems described.	Problem statement lists fewer than 3 problems or only superficially related to data management.	Clearly describes 3 data management problems but lacks depth or relevance in explanation.	Well-articulated problem identifies 3 relevant data management problems with reasonable detail	Comprehensive problem statements with at least 3 clearly defined, relevant and well-explained data management problems demonstrate clear understanding of impact	1	5	

Project Objectives	Objectives missing or unclear	Objectives vague or incomplete	Objectives stated but lacking detail	Clear and well-defined objectives	Precise, comprehensive and aligned with problems	1	5	
TOTAL							23	

ii) Database Design Rubric

Criteria	Score					Weight	Total / Section	MARK GIVEN
	1 (POOR)	2 (BELOW AVERAGE)	3 (AVERAGE)	4 (GOOD)	5 (EXCELLENT)	w		
Business Rules & ERD using Crow's Foot Notation	ERD is incomplete. Attributes are missing, keys are not identified, and relationships are incorrect or missing. Crow's Foot notation is not used or used incorrectly	ERD shows some entities, but many attributes or key attributes are missing or incorrectly marked. Relationships are poorly defined, and Crow's Foot notation is inconsistent applied	ERD has most entities with attributes and keys identified correctly. Relationships are mostly Correct with Crow's Foot notation used properly but some minor mistakes or omissions	ERD shows complete entities with attributes and keys clearly identified. Relationships are properly defined with correct use of Crow's Foot notation indicating cardinality and modality accurately. Minor details might be missing	ERD is fully complete with all attributes and keys correctly included. All relationships between entities are thoroughly defined and clearly shown with accurate Crow's Foot notation, including cardinality, modality and integrity constraints. Diagrams are neat and easy to understand	1	5	
Normalized ERD	ERD shows no signs of normalization. Tables have clear redundant fields or anomalies	Partial normalization but some redundant or repeated data remains	ERD shows basic normalization (1NF and 2NF) but may miss some refinements	ERD is mostly normalized with minimal redundancy. Most anomalies addressed	ERD fully normalized (up to 3NF as required), eliminating redundancy and ensuring data integrity	1	5	

Data Dictionary	Data dictionary is missing or incomplete with many entries omitted or incorrect	Data dictionary has partial entries; some attribute definitions or data types are unclear or incorrect	Data dictionary includes all entities and attributes but definitions or data types may be inconsistent	Data dictionary clearly defines entities, attributes and data types with few errors	Complete and well-organized data dictionary clearly defining all entities, attributes, data types, constraints and additional notes	1	5	
TOTAL							15	

iii) Database Implementation Rubric

Criteria	Score					Weight	Total / Section	MARK GIVEN
	1 (POOR)	2 (BELOW AVERAGE)	3 (AVERAGE)	4 (GOOD)	5 (EXCELLENT)	w		
Execution of DDL Statements (DDL – CREATE TABLE)	DDL statements are mostly incorrect or missing	Some correct statements but with major errors	Basic correct DDL commands with minor mistakes.	Accurate statements with few minor issues.	Fully correct, well-structured DDL with all required elements.	1	5	
Execution of DML for Single Table Data Retrieval (BASIC SELECT)	DML queries incorrect or missing	Simple queries with many syntax or logic errors	Queries retrieve data with minor mistakes or inefficiencies	Correct queries that retrieve relevant data properly	Efficient and accurate queries demonstrating good SQL use	1	5	
Execution of DML for Multi-table Retrieval Using Different Relationship Clauses (JOIN)	No multi-table queries or completely incorrect	Queries attempt joins but contain errors	Basic join queries with some minor issues	Correct join queries covering multiple relationship types	Complex, accurate joins demonstrating thorough understanding	1	5	

Execution of DML Using Comprehensive Aggregate Functions	Functions missing or used incorrectly	Basic use with errors	Correct use of some scalar or aggregate functions	Efficient use of column functions covering multiple cases	Advanced and correct use of functions optimizing queries	2	10	
Execution of DML Using Comprehensive Subqueries	No subqueries or incorrect usage.	Attempts at subqueries but with errors.	Functionally correct subqueries with minor flaws	Accurate and effective use of subqueries.	Complex and well-optimized subqueries demonstrating deep understanding.	1	5	
Execution of DML Using Comparison Operators for Data Filtering (DML – UPDATE & DELETE)	No use or incorrect use of comparison operators.	Basic filtering but syntax or logic errors present.	Functional use of comparison operators with minor errors.	Correct and efficient use of operators for filtering.	Sophisticated filtering demonstrating mastery of operators.	1	5	
Relevance of Record in Each Table for Data Manipulation (DML – INSERT)	Records are mostly irrelevant or unrelated to the table's purpose; data manipulation queries result in incorrect or meaningless outputs.	Many records are relevant but there are significant irrelevant or redundant entries affecting query outcomes.	Most records are relevant and support intended data operations; minor inconsistencies may exist.	Records are carefully selected and consistent, enabling accurate and meaningful data manipulation with negligible irrelevant data.	All records are highly relevant and precisely aligned with the table's role; queries produce consistently correct and valuable results.	0.6	3	
TOTAL							38	
TOTAL MARK FOR PROJECT REPORT							76	